

Digital Design & Computer Arch.

Lecture 26: Virtual Memory

Konstantinos Kanellopoulos
Prof. Onur Mutlu

ETH Zürich
Spring 2026
29 May 2026

A Blueprint for Fundamentally Better Architectures

- Onur Mutlu,
"Intelligent Architectures for Intelligent Computing Systems"
*Invited Paper in Proceedings of the Design, Automation, and Test in Europe Conference (**DATE**), Virtual, February 2021.*
[[Slides \(pptx\)](#)] [[pdf](#)]
[[IEDM Tutorial Slides \(pptx\)](#)] [[pdf](#)]
[[Short DATE Talk Video](#) (11 minutes)]
[[Longer IEDM Tutorial Video](#) (1 hr 51 minutes)]

Intelligent Architectures for Intelligent Computing Systems

Onur Mutlu
ETH Zurich
omutlu@gmail.com

Fundamentally Better Architectures

Data-centric

Data-driven

Data-aware

Readings

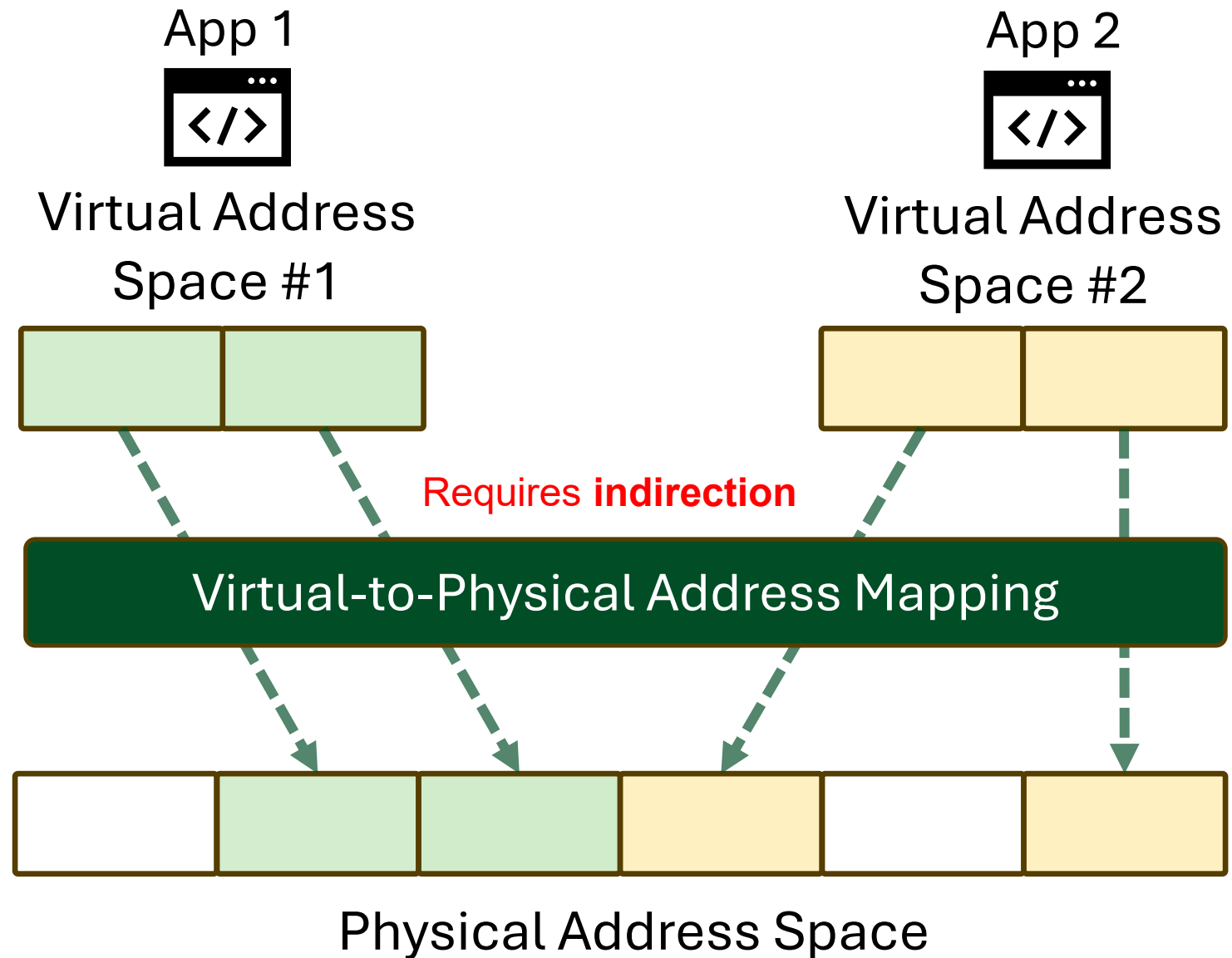
■ Virtual Memory

■ Recommended

- H&H Chapter 8.4
- Kim & Mutlu, “**Memory Systems**,” Computing Handbook, 2014.
 - https://people.inf.ethz.ch/omutlu/pub/memory-systems-introduction_computing-handbook14.pdf
- Jacob & Mudge, “**Virtual Memory: Issues of Implementation**,” IEEE Computer, 1998.
- Kanellopoulos et al. “**Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings**” MICRO 2023
- Kanellopoulos et al. “**Virtuoso: Enabling Fast and Accurate Virtual Memory Research via an Imitation-based Operating System Simulation Methodology**” ASPLOS 2025
- Hajinazar et al. “**The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework**” ISCA 2020

Benefits of Automatic Management of Memory

- Programmer does not deal with physical addresses
- Each process has its own
 - Virtual address space (very large)
 - Independent mapping of virtual→physical addresses
- Enables
 - Code and data to be located anywhere in physical memory
(relocation and flexible location of data)
 - Isolation/separation of code and data of different processes in physical memory
(protection and isolation)
 - Code and data sharing between multiple processes
(sharing)

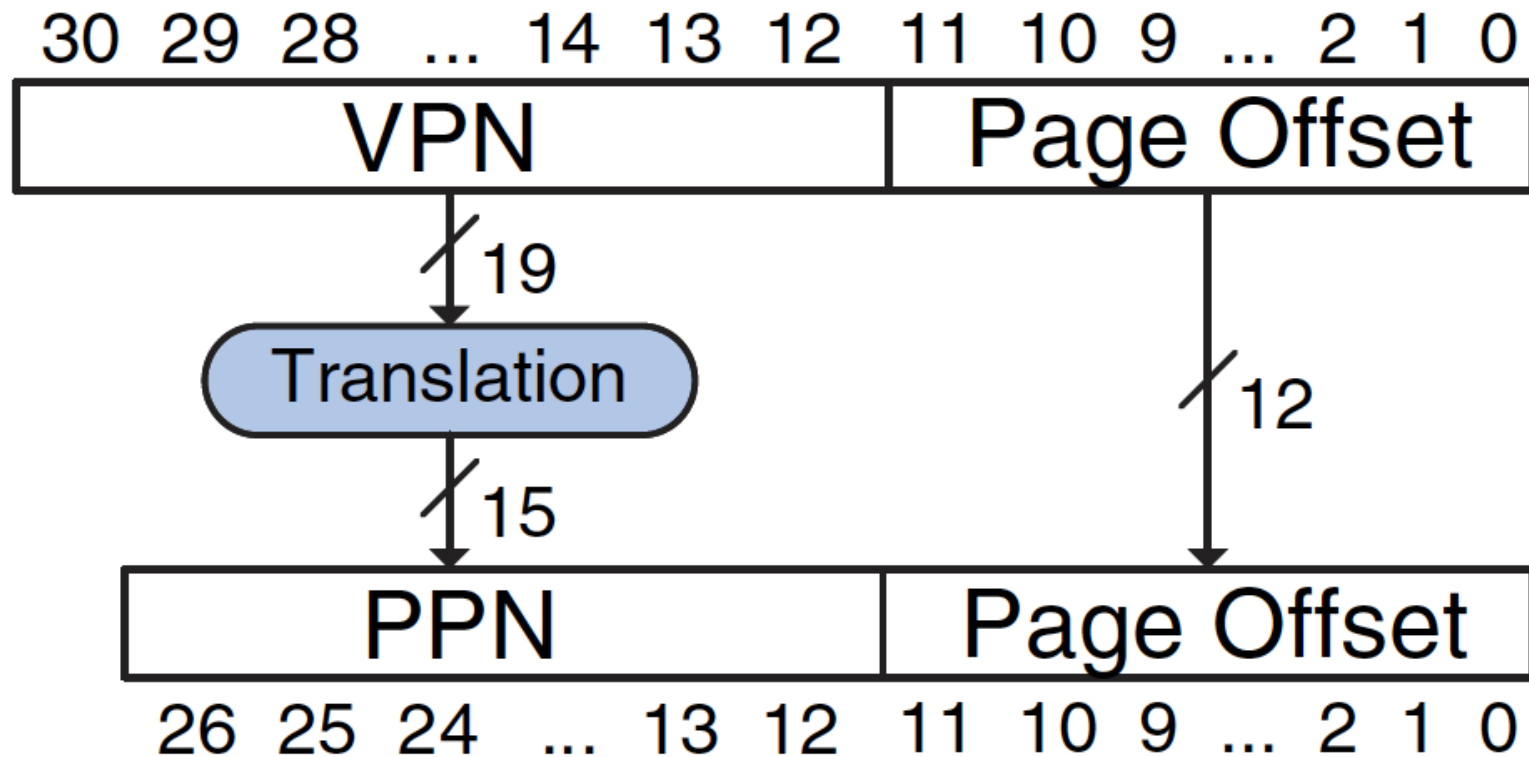


Four Issues in Indirection and Mapping

- When to map a virtual address to a physical address?
 - When the virtual address is first referenced by the program
- What is the mapping granularity?
 - Byte? Kilo-byte? Mega-byte? Giga-byte? ...
 - Multiple granularities?
- Where and how to store the virtual→physical mappings?
 - Operating system data structures? Hardware? Cooperative?
- What to do when physical address space is full?
 - Evict an unlikely-to-be-needed virtual address from physical memory

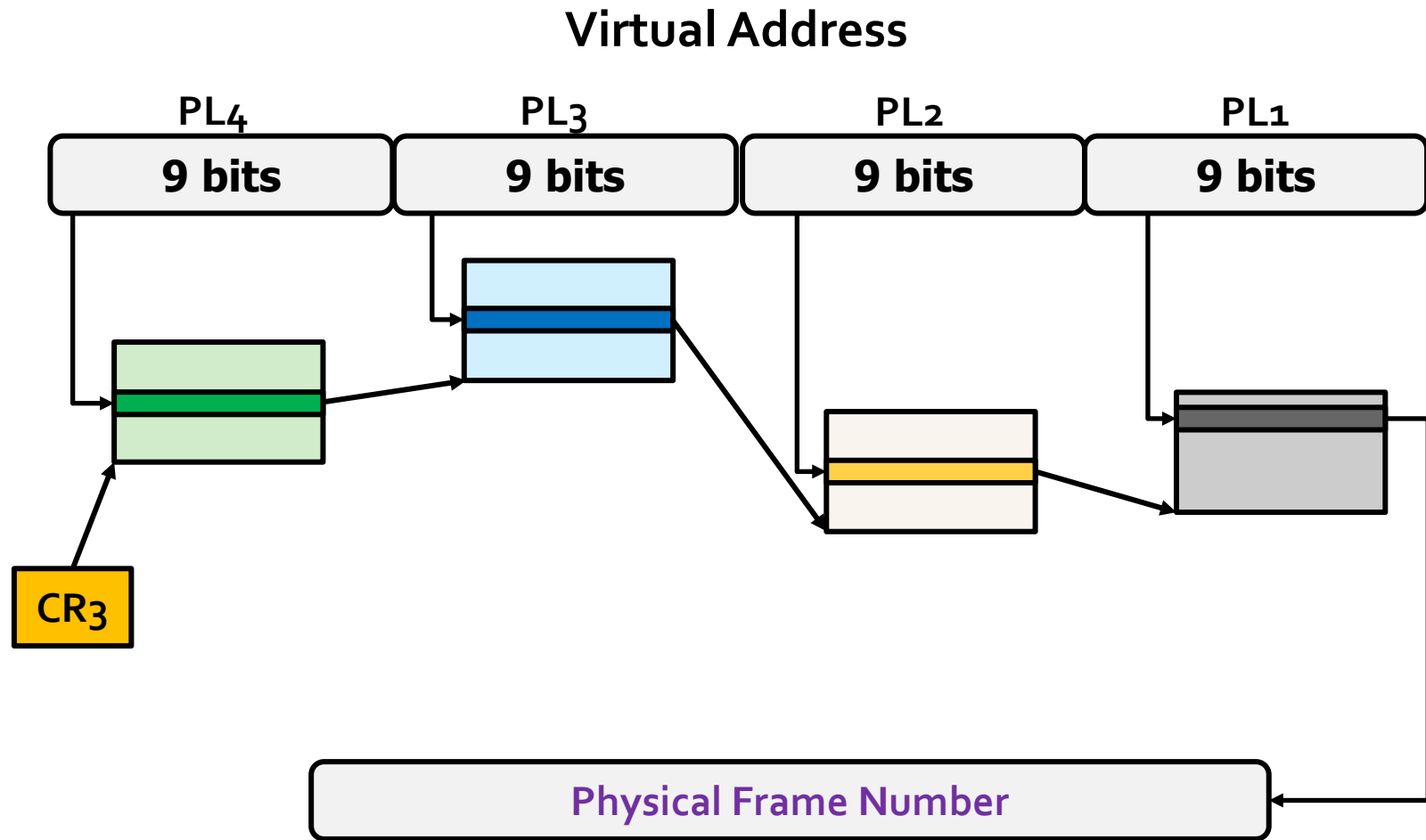
Address Translation

Virtual Address

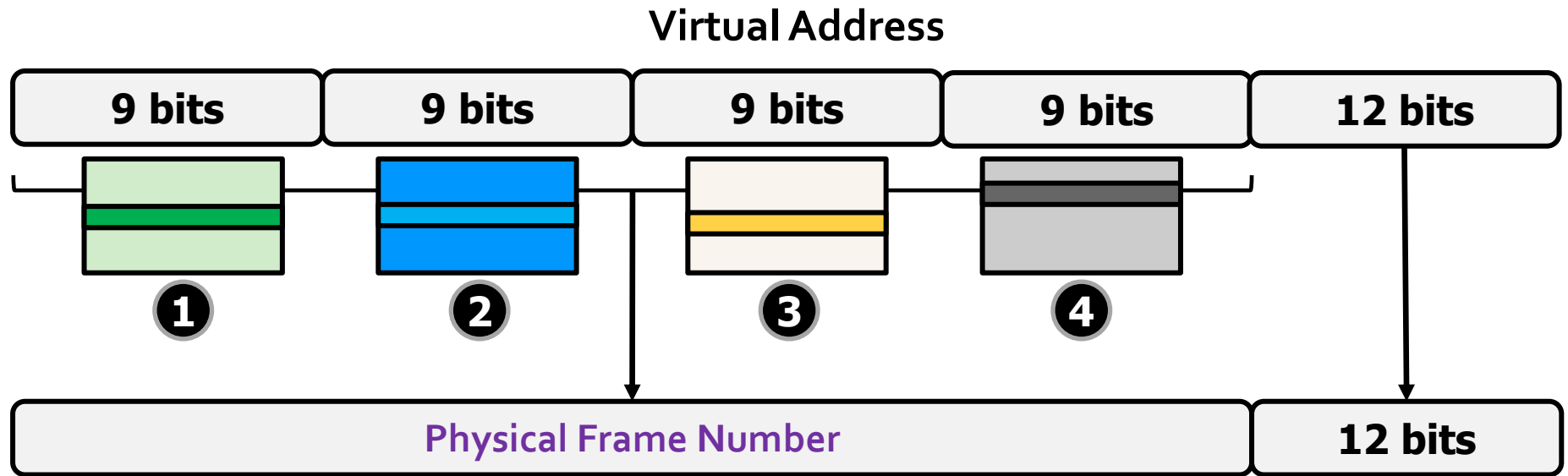


Physical Address

Page Table in x86-64



Page Table Walk in x86-64



Four sequential memory accesses during a page table walk in x86-64

How does the CPU discover the
virtual-to-physical mapping?

Servicing a Major Page Fault

1. Processor signals I/O controller

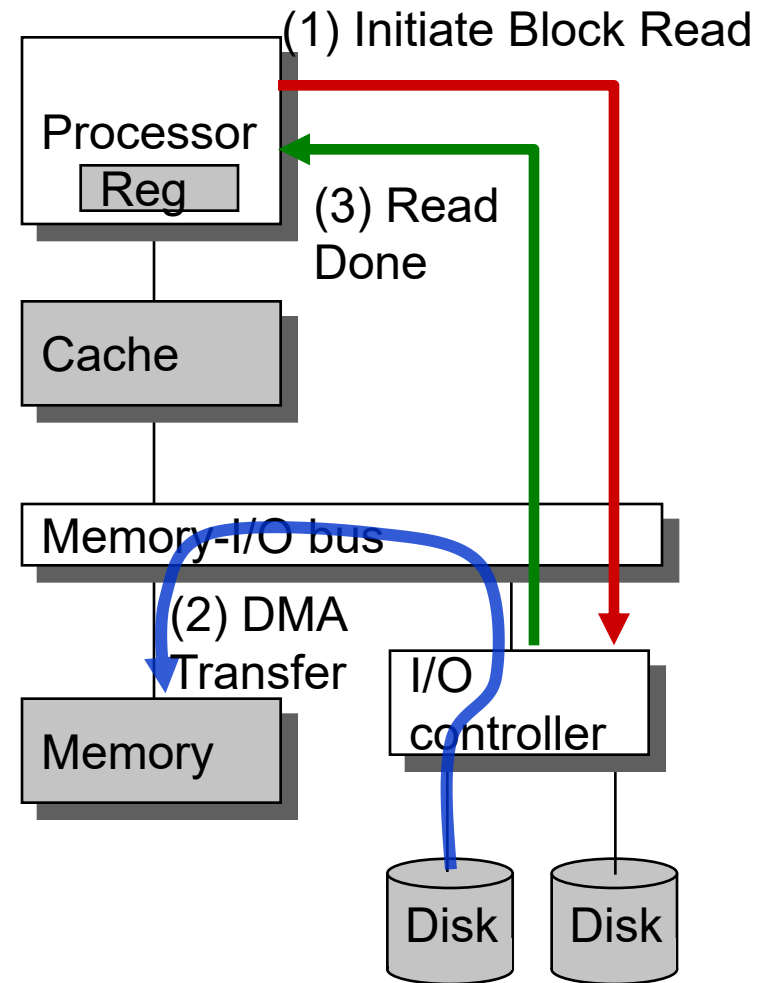
- Read block of length P starting at disk address X and store starting at memory address Y

2. Disk-to-mem read occurs

- Direct Memory Access (DMA)
- Under control of I/O controller

3. Controller signals completion

- Interrupts processor
- OS resumes suspended process

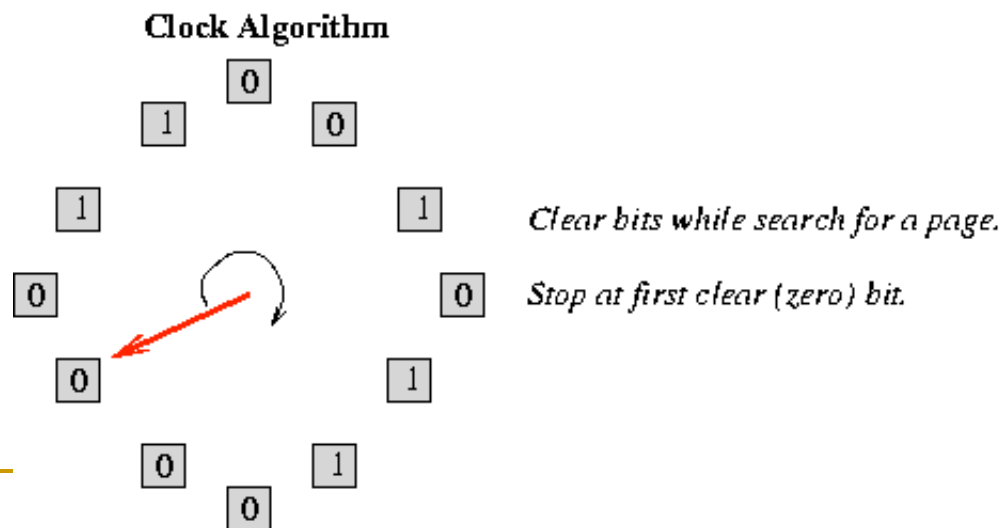


Page Replacement Algorithms

- If physical memory is full (i.e., list of free physical pages is empty), which physical frame to replace on a page fault?
- Is True LRU feasible?
 - 4GB memory, 4KB pages, how many possibilities of ordering?
- Modern systems use approximations of LRU
 - E.g., the CLOCK algorithm
- And, more sophisticated algorithms to take into account “frequency” of use
 - E.g., the ARC algorithm
 - Megiddo and Modha, “ARC: A Self-Tuning, Low Overhead Replacement Cache,” FAST 2003.

CLOCK Page Replacement Algorithm

- Keep a **circular list of physical frames** in memory (OS does)
- Keep a **pointer** (hand) to the last-examined frame in the list
- When a page is accessed, set the R bit in the PTE
- When a frame needs to be replaced, replace the first frame that has the reference (R) bit not set, traversing the circular list starting from the pointer (hand) clockwise
 - ❑ During traversal, clear the R bits of examined frames
 - ❑ Set the hand pointer to the next frame in the list



Cache versus Page Replacement

- Physical memory (DRAM) is a cache for disk
 - Managed by system software via the virtual memory subsystem
- Page replacement is similar to cache replacement
- Page table is the “tag store” for physical memory data store
- What is the difference?
 - Required speed of access to cache vs. physical memory
 - Number of blocks in a cache vs. physical memory
 - “Tolerable” amount of time to find a replacement candidate (disk versus memory access latency)
 - Role of hardware versus software

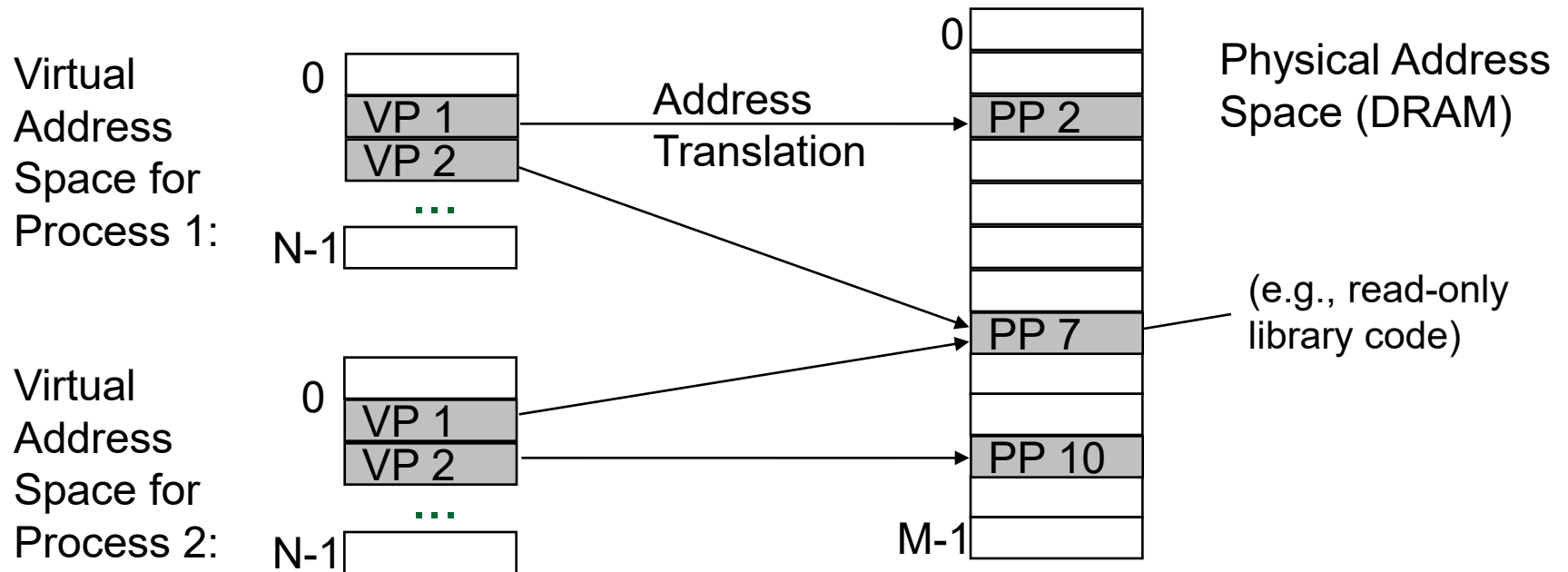
Memory Protection

Memory Protection

- Multiple programs (i.e., processes) run concurrently
 - Each process has its own page table
 - Each process can use its entire virtual address space without worrying about where other programs are
- A process can only access physical pages mapped in its page table – cannot overwrite memory of another process
 - Provides protection and isolation between processes
 - Enables access control mechanisms per page

Page Table is Per Process

- Each process has its own virtual address space
 - Full address space for each program
 - Simplifies memory allocation, sharing, linking and loading

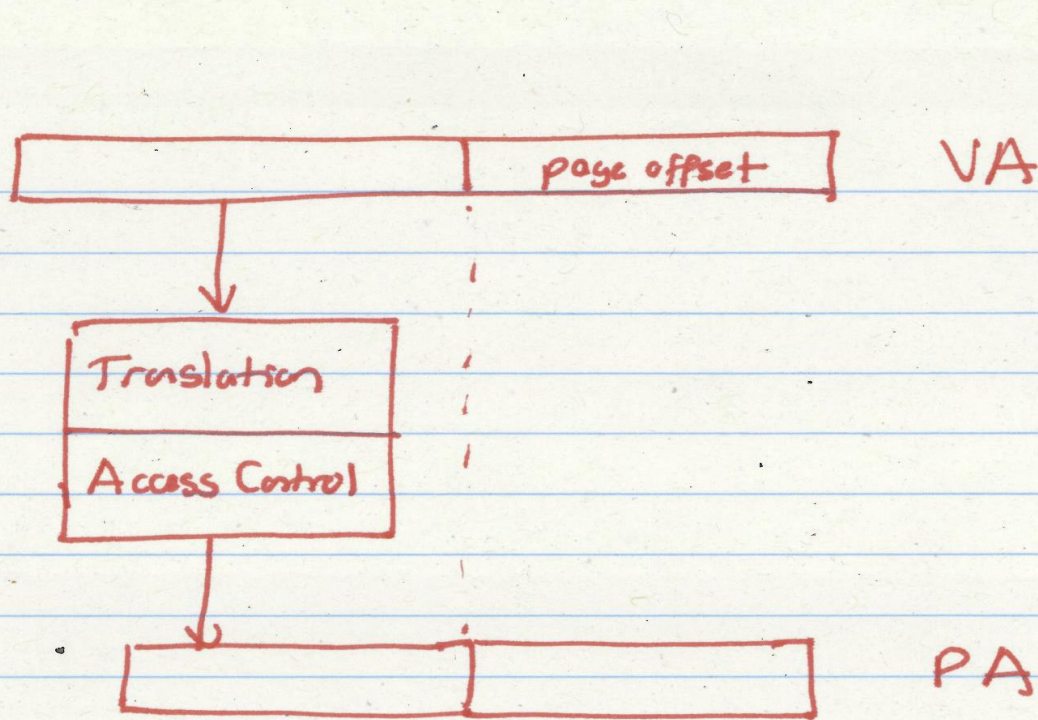


Access Protection/Control via Virtual Memory

Page-Level Access Control (Protection)

- Not every process is allowed to access every page
 - E.g., need supervisor (i.e., kernel) level privilege to access system pages
 - E.g., may not be able to execute “instructions” in some pages
 - Idea: Store access control information on a page basis in the process's page table
 - Enforce access control at the same time as translation
- Virtual memory system serves two functions today
- Address translation (for illusion of large physical memory)
 - Access control (protection)

Two Functions of Virtual Memory



Virtual
Memory

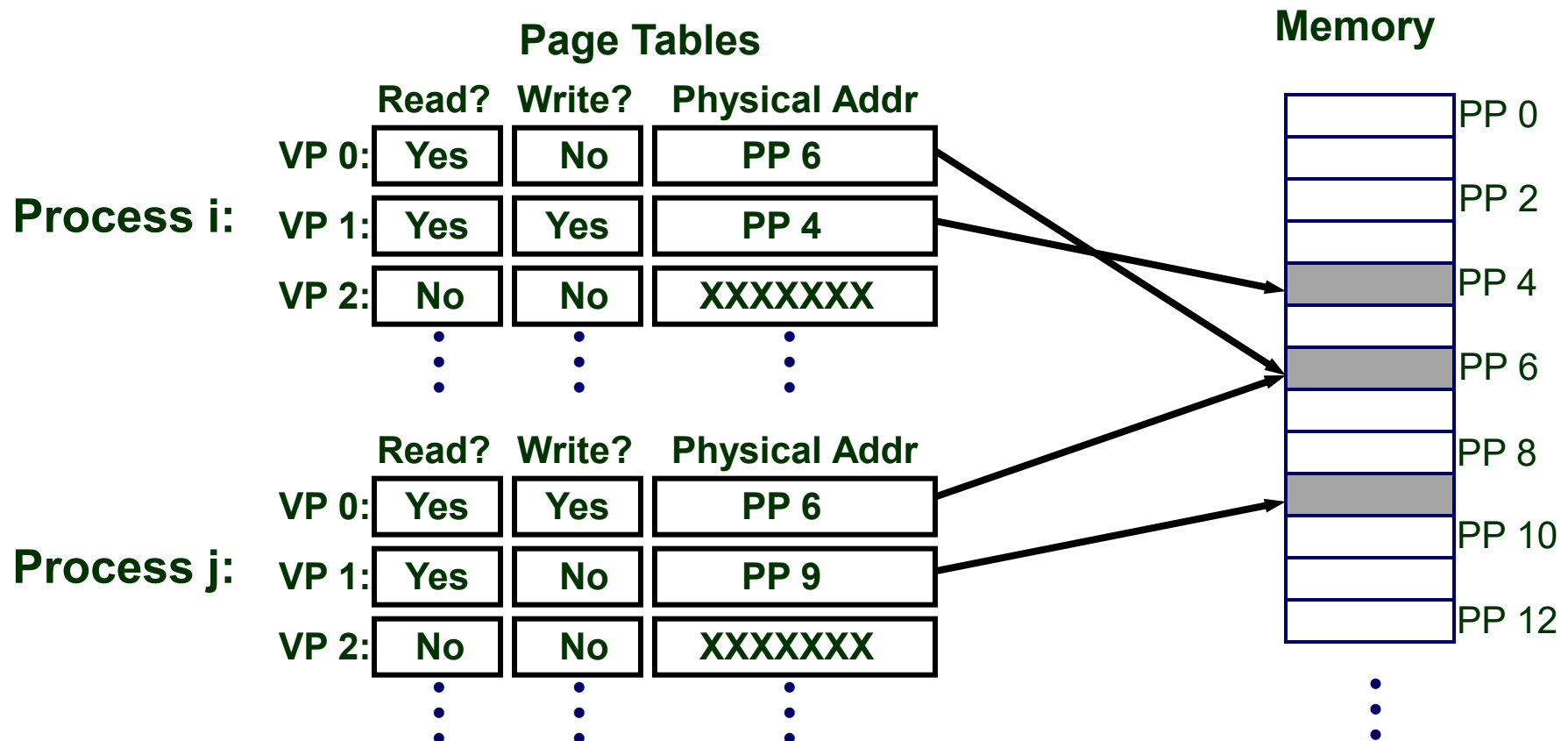
Two Functions
Today

1. Translation
2. Access control (protection)

PTE contains access control bits associated with the virtual page.

VM as a Tool for Memory Access Protection

- Extend Page Table Entries (PTEs) with permission bits
- Check bits on each access and during a page fault
 - If violated, generate exception (Access Protection exception)



Privilege Levels in x86

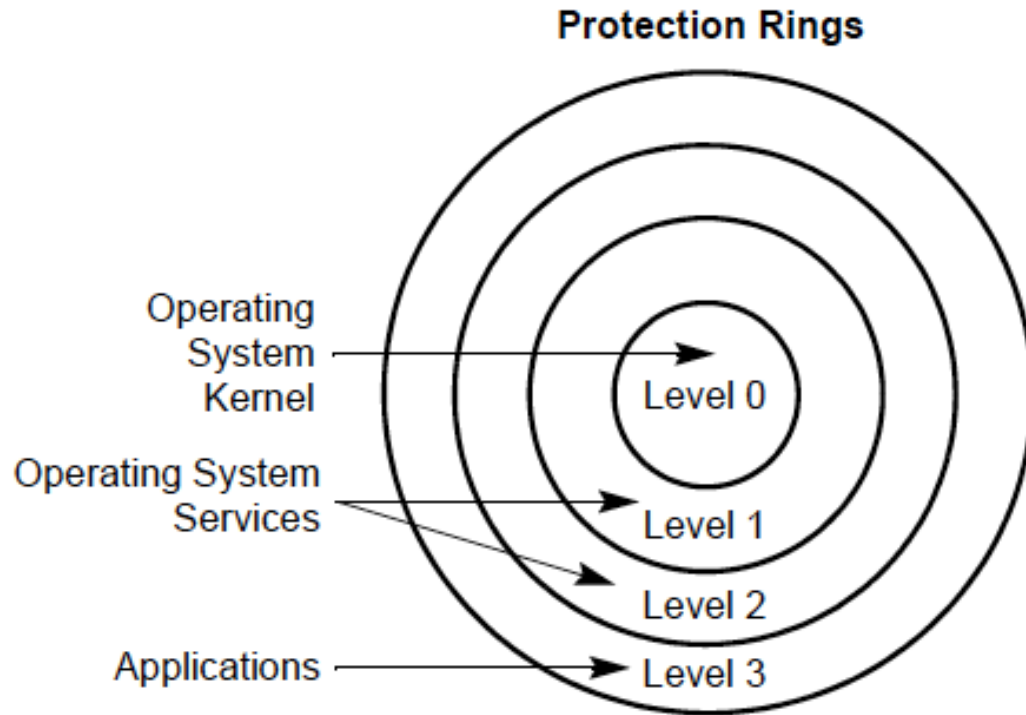


Figure 5-3. Protection Rings

Privilege Levels in x86

- Four **privilege levels** in x86 (referred to as **rings**)

- Ring 0: Highest privilege (operating system)
 - Ring 1: Not widely used
 - Ring 2: Not widely used
 - Ring 3: Lowest privilege (user applications)
- "Supervisor"**
- "User"**

- Supervisor = Kernel (in modern terminology)
-

x86: A Closer Look at the PDE/PTE

- **PDE:** Page Directory Entry (32 bits)
- **PTE:** Page Table Entry (32 bits)

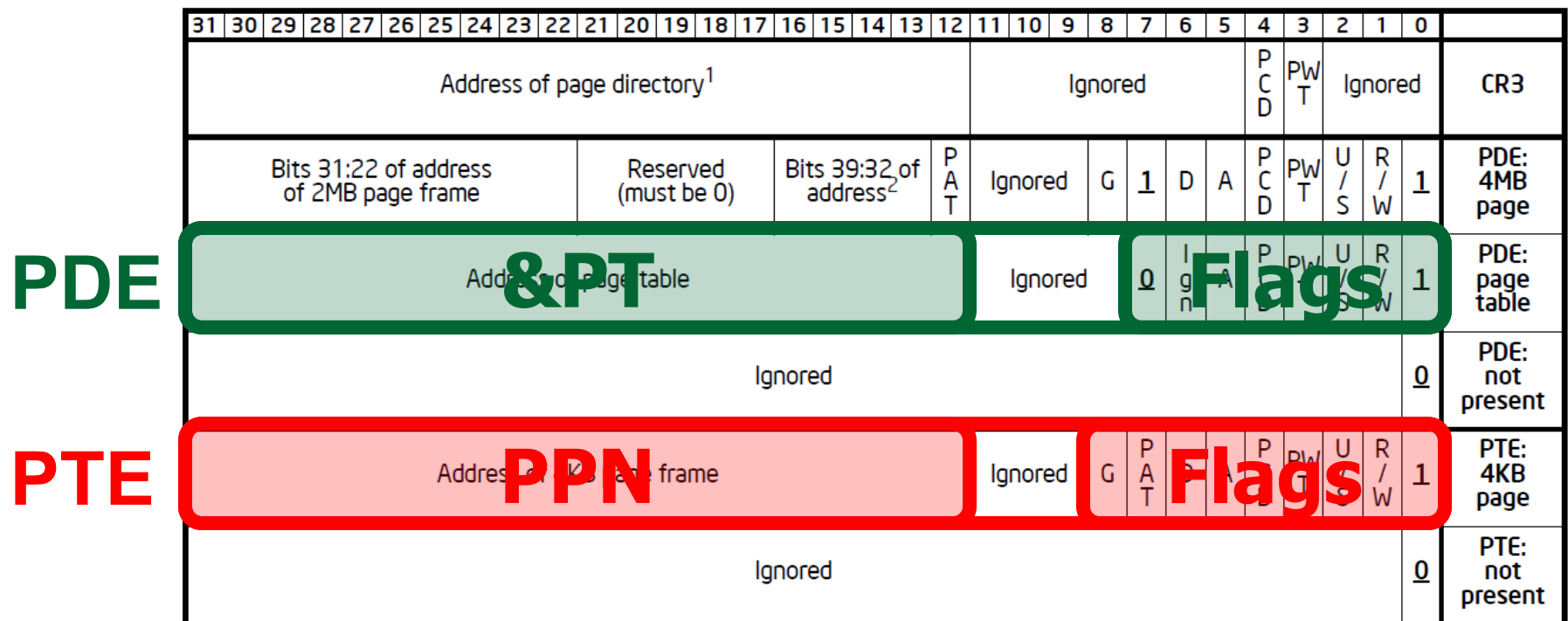


Figure 4-4. Formats of CR3 and Paging-Structure Entries with 32-Bit Paging

Protection: PDE's Flags

- Protects all 1024 pages in a page table

Table 4-5. Format of a 32-Bit Page-Directory Entry that References a Page Table

Bit Position(s)	Contents
0 (P)	Present; must be 1 to reference a page table
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-MByte region controlled by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-MByte region controlled by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether this entry has been used for linear-address translation (see Section 4.8)
6	Ignored
7 (PS)	If CR4.PSE = 1, must be 0 (otherwise, this entry maps a 4-MByte page; see Table 4-4); otherwise, ignored

Protection: PTE's Flags

■ Protects one page at a time

Table 4-6. Format of a 32-Bit Page-Table Entry that Maps a 4-KByte Page

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0) ¹
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise

Page Level Protection in x86

Table 5-3. Combined Page-Directory and Page-Table Protection

Page-Directory Entry		Page-Table Entry		Combined Effect	
Privilege	Access Type	Privilege	Access Type	Privilege	Access Type
User	Read-Only	User	Read-Only	User	Read-Only
User	Read-Only	User	Read-Write	User	Read-Only
User	Read-Write	User	Read-Only	User	Read-Only
User	Read-Write	User	Read-Write	User	Read/Write
User	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write ⁺
User	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write ⁺
User	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write ⁺
User	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	User	Read-Only	Supervisor	Read/Write ⁺
Supervisor	Read-Only	User	Read-Write	Supervisor	Read/Write ⁺
Supervisor	Read-Write	User	Read-Only	Supervisor	Read/Write ⁺
Supervisor	Read-Write	User	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write ⁺
Supervisor	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write ⁺
Supervisor	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write ⁺
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write

Protection: PDE + PTE = ???

Table 5-3. Combined Page-Directory and Page-Table Protection

Page-Directory Entry		Page-Table Entry		Combined Effect	
Privilege	Access Type	Privilege	Access Type	Privilege	Access Type
User	Read-Only	User	Read-Only	User	Read-Only
User	Read-Only	User	Read-Write	User	Read-Only
User	Read-Write	User	Read-Only	User	Read-Only
User	Read-Write	User	Read-Write	User	Read/Write
User	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	User	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write

NOTE:

* If CR0.WP = 1, access type is determined by the R/W flags of the page-directory and page-table entries. If CR0.WP = 0, supervisor privilege permits read-write access.

Food for Thought: What If?

- Your hardware is unreliable and someone can flip the access protection bits
 - such that a user-level program can gain supervisor-level access (i.e., access to all data on the system)
 - by flipping the access control bit from user to supervisor!

- Can this happen?

Remember RowHammer?

One can
predictably induce errors
in most DRAM memory chips

Remember RowHammer?

- One can **predictably induce bit flips** in commodity DRAM chips
 - >80% of the tested DRAM chips are vulnerable
- First example of how a **simple hardware failure mechanism** can create a **widespread system security vulnerability**

WIRED

Forget Software—Now Hackers Are Exploiting Physics

BUSINESS	CULTURE	DESIGN	GEAR	SCIENCE
----------	---------	--------	------	---------

ANDY GREENBERG SECURITY 08.31.16 7:00 AM

SHARE



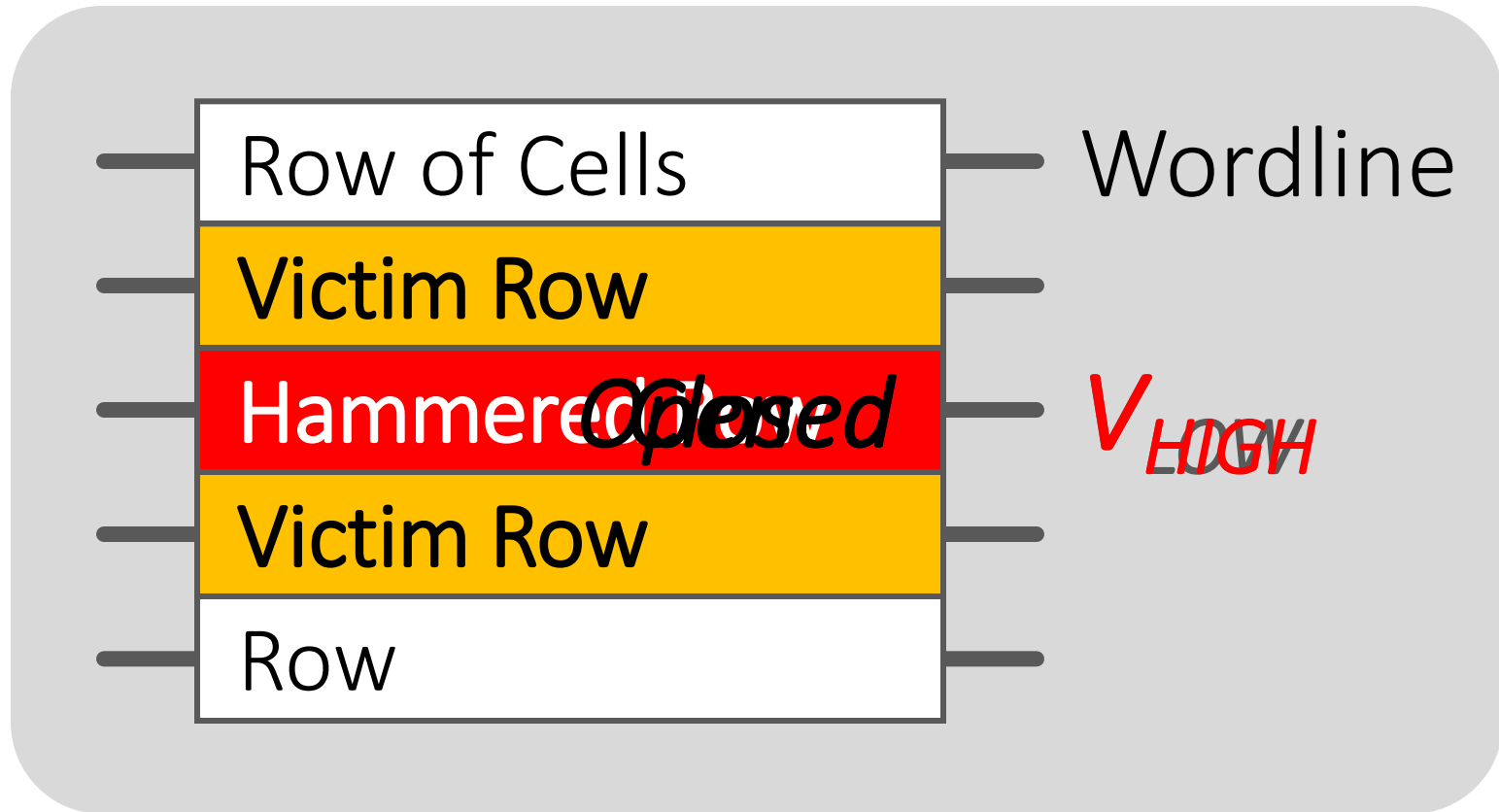
SHARE
18276



TWEET

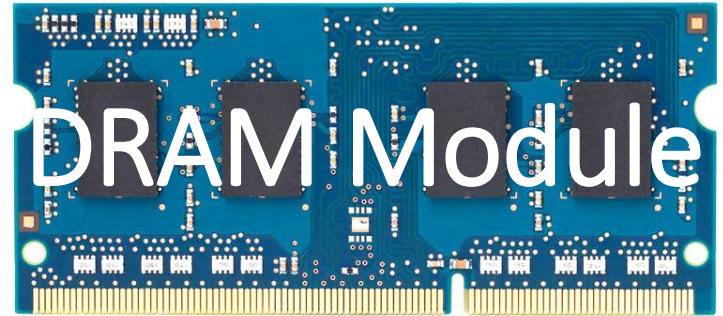
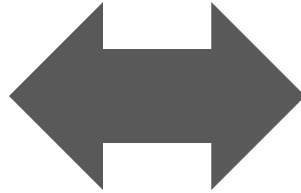
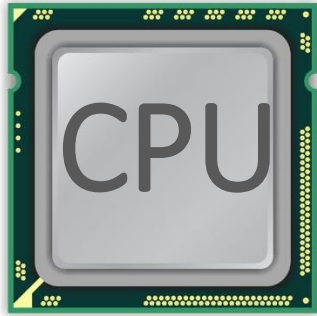
FORGET SOFTWARE—NOW HACKERS ARE EXPLOITING PHYSICS

Modern DRAM is Prone to Disturbance Errors

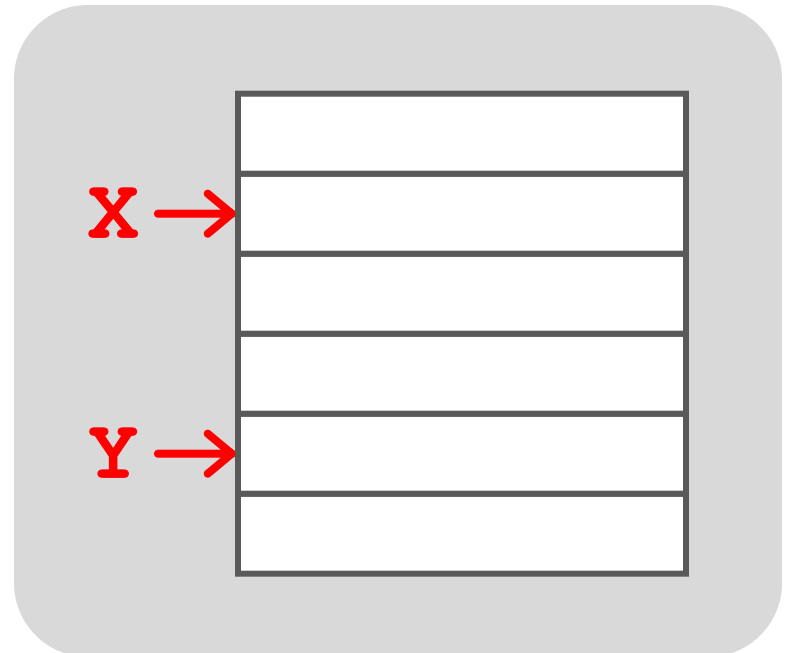


Repeatedly reading a row enough times (before memory gets refreshed) induces **disturbance errors** in **adjacent rows** in **most real DRAM chips you can buy today**

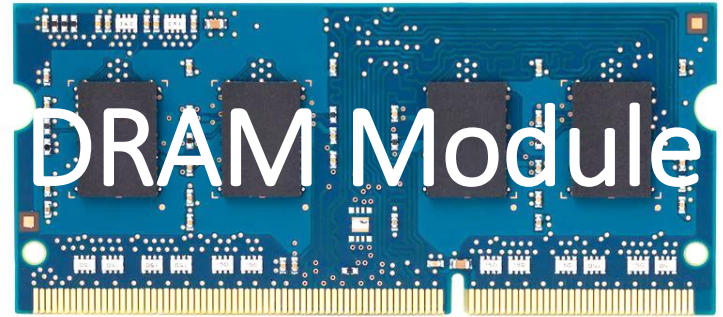
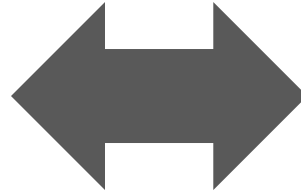
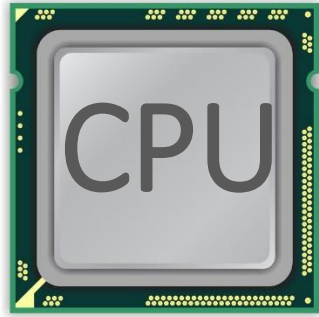
A Simple Program Can Induce Many Errors



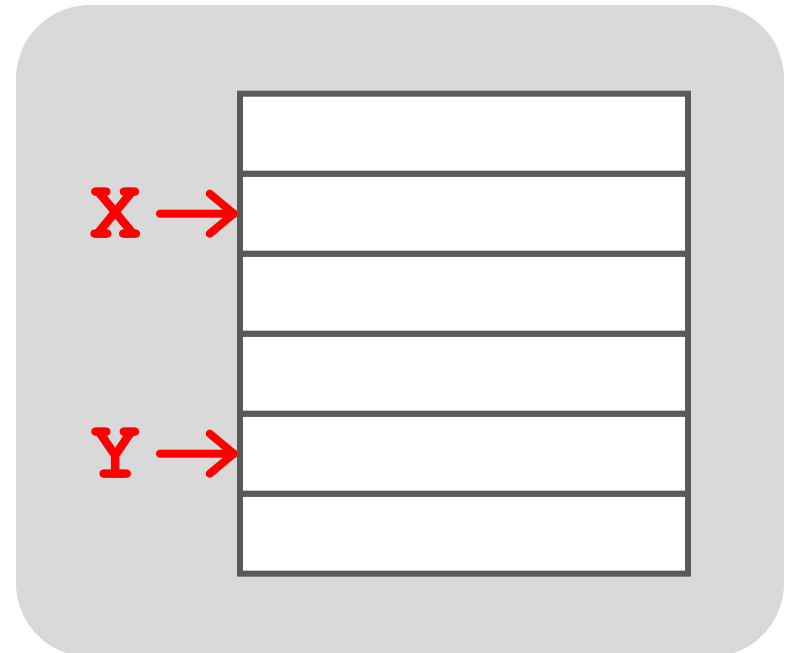
```
loop:  
  mov  (X), %eax  
  mov  (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp  loop
```



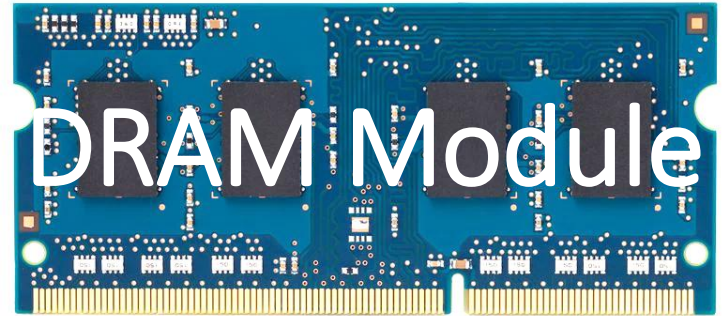
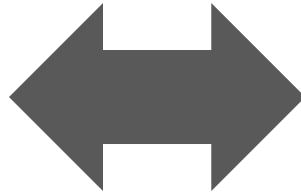
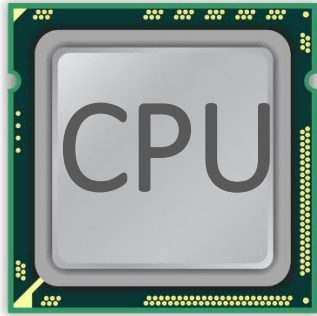
A Simple Program Can Induce Many Errors



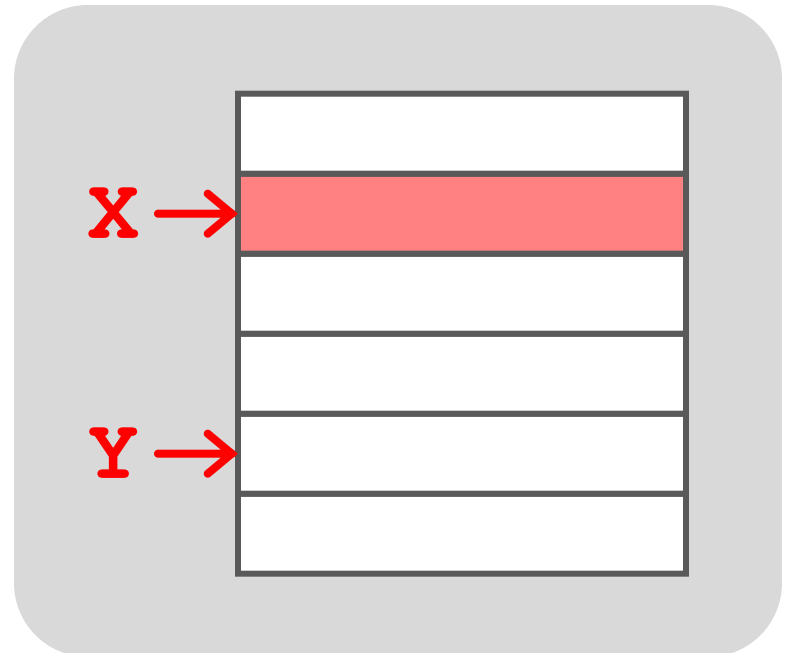
1. Avoid *cache hits*
 - Flush **X** from cache
2. Avoid *row hits* to **X**
 - Read **Y** in another row



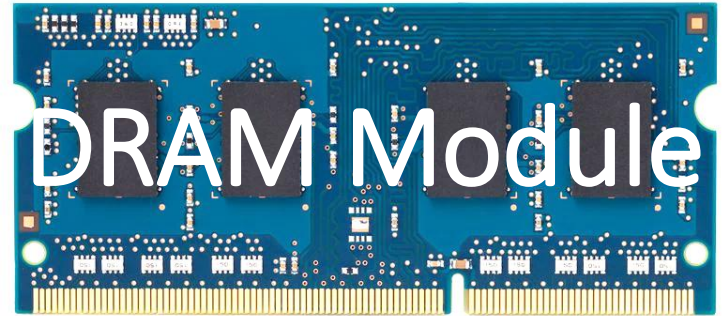
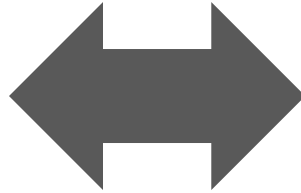
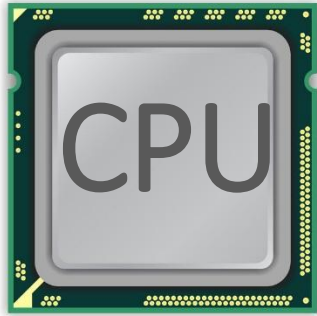
A Simple Program Can Induce Many Errors



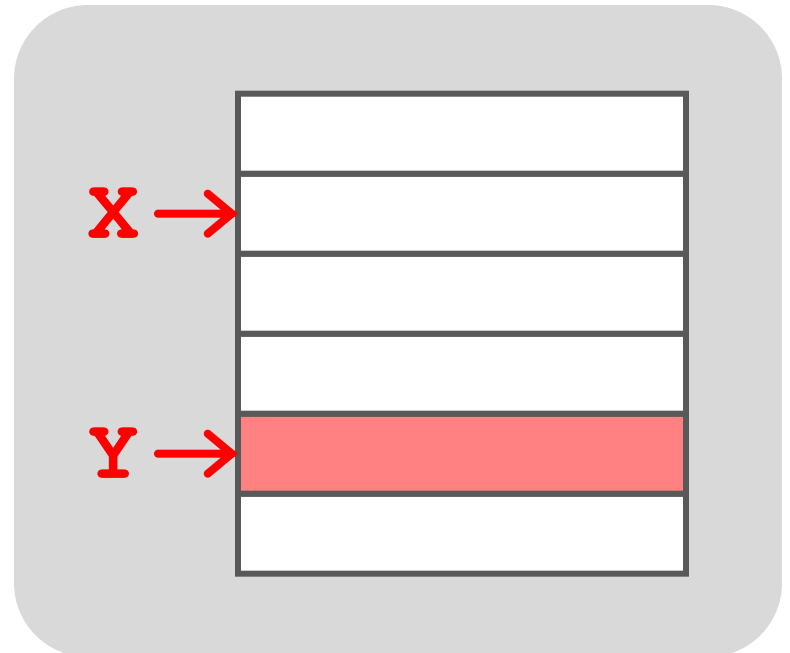
```
loop:  
  mov  (X), %eax  
  mov  (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp  loop
```



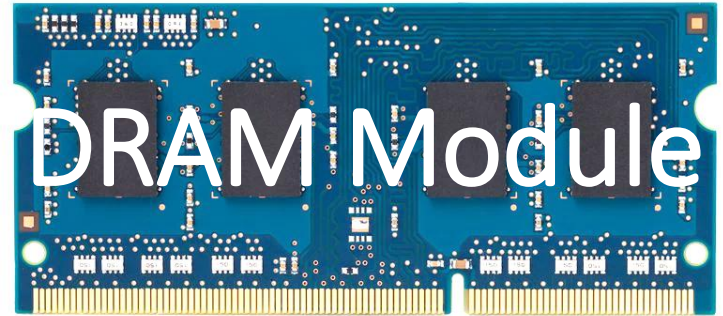
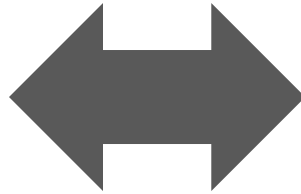
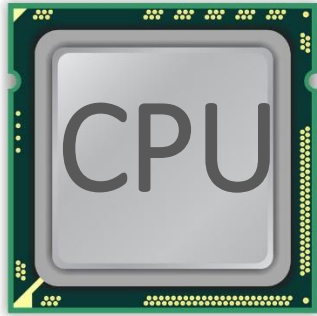
A Simple Program Can Induce Many Errors



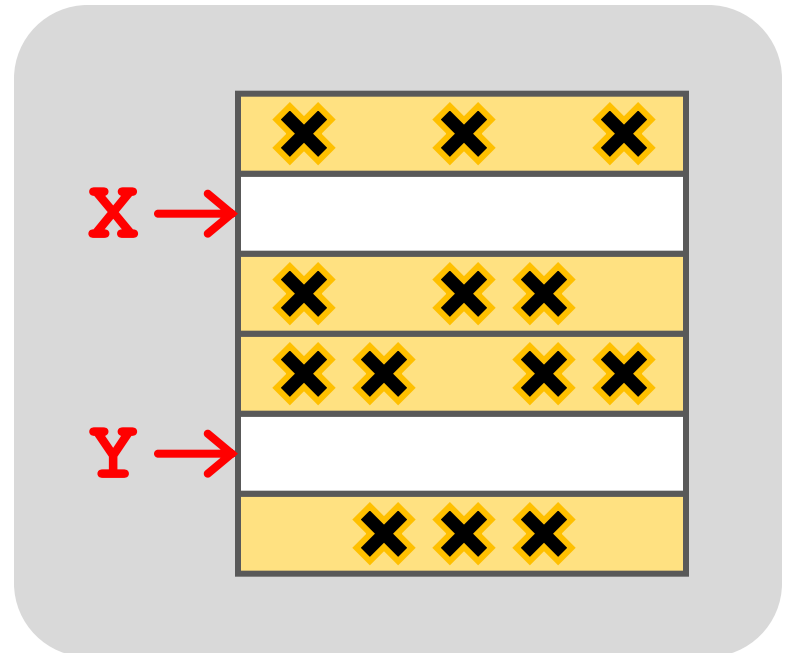
```
loop:  
  mov  (X), %eax  
  mov  (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp  loop
```



A Simple Program Can Induce Many Errors



```
loop:  
  mov  (X), %eax  
  mov  (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp  loop
```



Observed Errors in Real Systems

CPU Architecture	Errors	Access-Rate
Intel Haswell (2013)	22.9K	12.3M/sec
Intel Ivy Bridge (2012)	20.7K	11.7M/sec
Intel Sandy Bridge (2011)	16.1K	11.6M/sec
AMD Piledriver (2012)	59	6.1M/sec

A real reliability & security issue

One Can Take Over an Otherwise-Secure System

Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors

Abstract. Memory isolation is a key property of a reliable and secure computing system — an access to one memory address should not have unintended side effects on data stored in other addresses. However, as DRAM process technology

Project Zero

Flipping Bits in Memory Without Accessing Them:
An Experimental Study of DRAM Disturbance Errors
(Kim et al., ISCA 2014)

News and updates from the Project Zero team at Google

Exploiting the DRAM rowhammer bug to
gain kernel privileges (Seaborn, 2015)

Monday, March 9, 2015

Exploiting the DRAM rowhammer bug to gain kernel privileges

RowHammer Security Attack Example

- “Rowhammer” is a problem with some recent DRAM devices in which repeatedly accessing a row of memory can cause bit flips in adjacent rows (Kim et al., ISCA 2014).
 - Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors (Kim et al., ISCA 2014)
- We tested a selection of laptops and found that a subset of them exhibited the problem.
- We built two working privilege escalation exploits that use this effect.
 - Exploiting the DRAM rowhammer bug to gain kernel privileges (Seaborn+, 2015)
- One exploit uses rowhammer-induced bit flips to gain kernel privileges on x86-64 Linux when run as an unprivileged userland process.
- When run on a machine vulnerable to the rowhammer problem, the process was able to induce bit flips in page table entries (PTEs).
- It was able to use this to gain write access to its own page table, and hence gain read-write access to all of physical memory.

Google's Original RowHammer Attack

The following slides are from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>

<https://www.youtube.com/watch?v=0U7511Fb4to>

Kernel exploit

- x86 page tables entries (PTEs) are **dense and trusted**
 - They control access to physical memory
 - A bit flip in a PTE's physical page number can give a process access to a different physical page
- Aim of exploit: Get access to a page table
 - Gives access to all of physical memory
- Maximise chances that a bit flip is useful:
 - Spray physical memory with page tables
 - Check for useful, repeatable bit flip first

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>

x86-64 Page Table Entries (PTEs)

- Page table is a 4k page containing array of 512 PTEs
- Each PTE is 64 bits, containing:

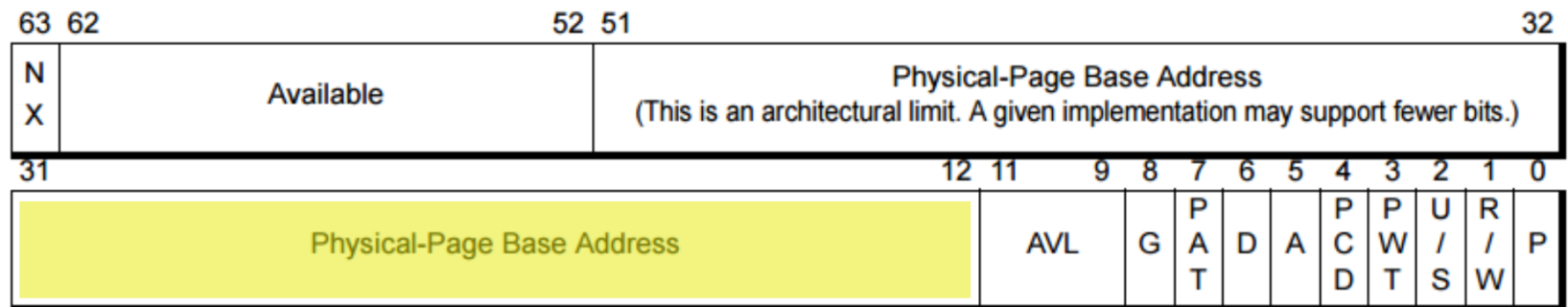
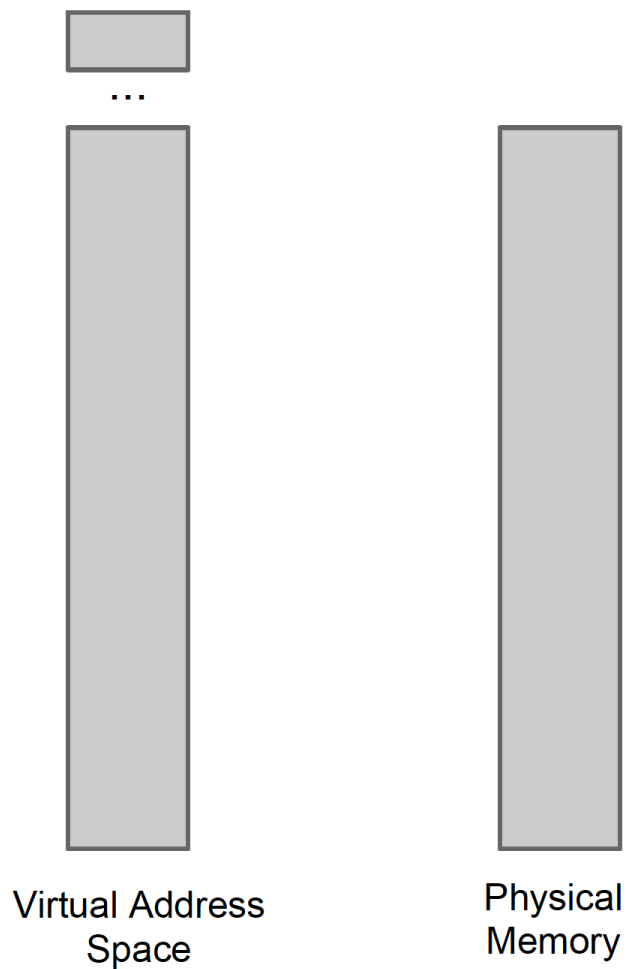


Figure 5-21. 4-Kbyte PTE—Long Mode

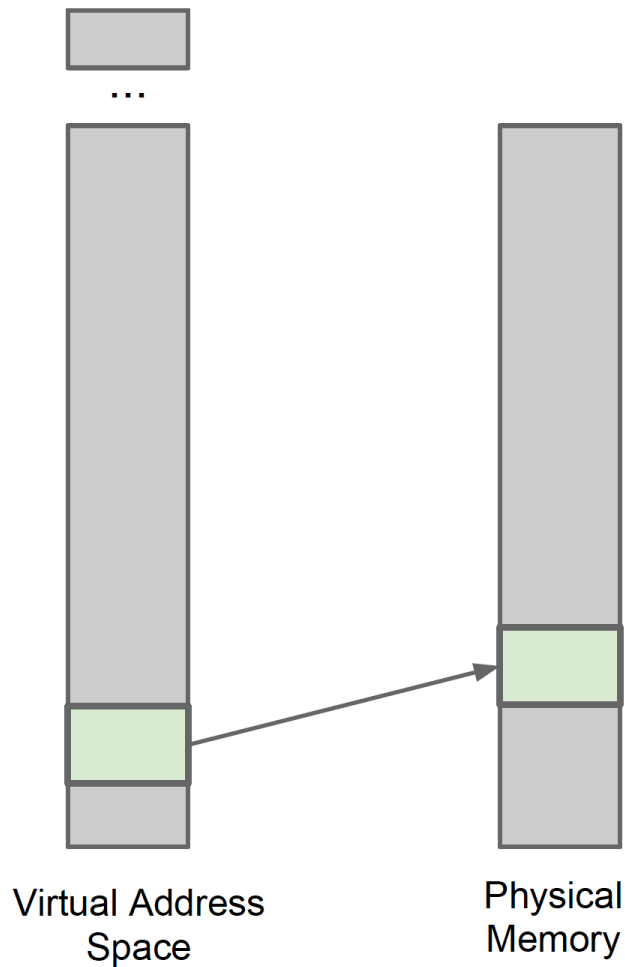
- Could flip:
 - “Writable” permission bit (RW): 1 bit → 2% chance
 - Physical page number: 20 bits on 4GB system → 31% chance

This slide is from Mark Seaborn and Thomas Dullien’s BlackHat 2015 talk



This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

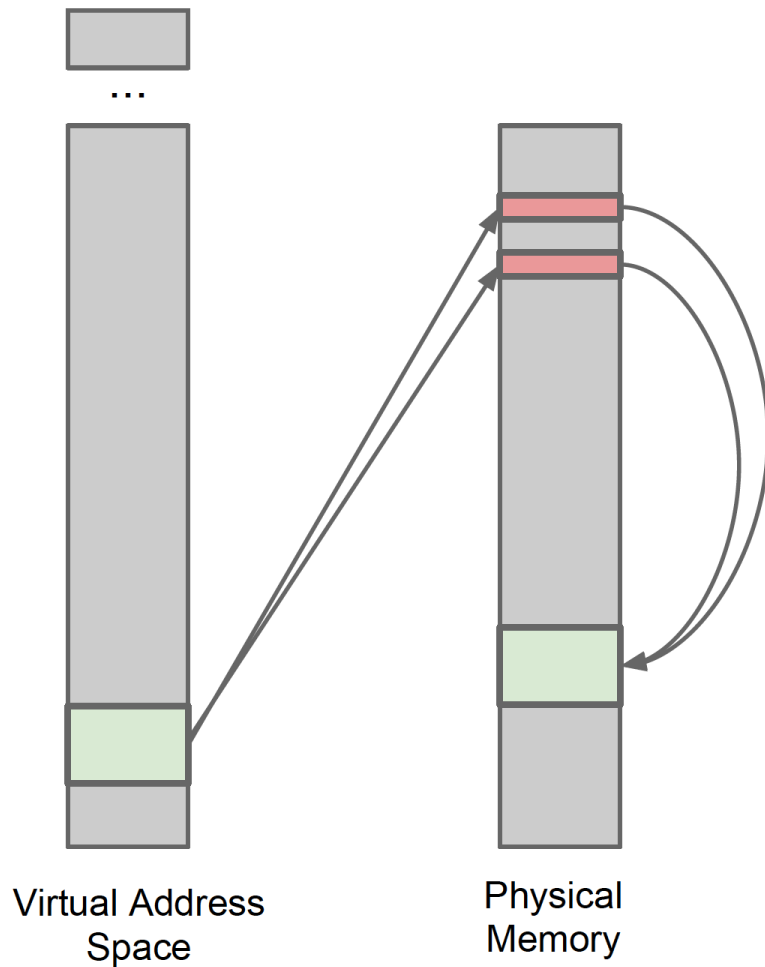
<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>



What happens when we map a file with read-write permissions?

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

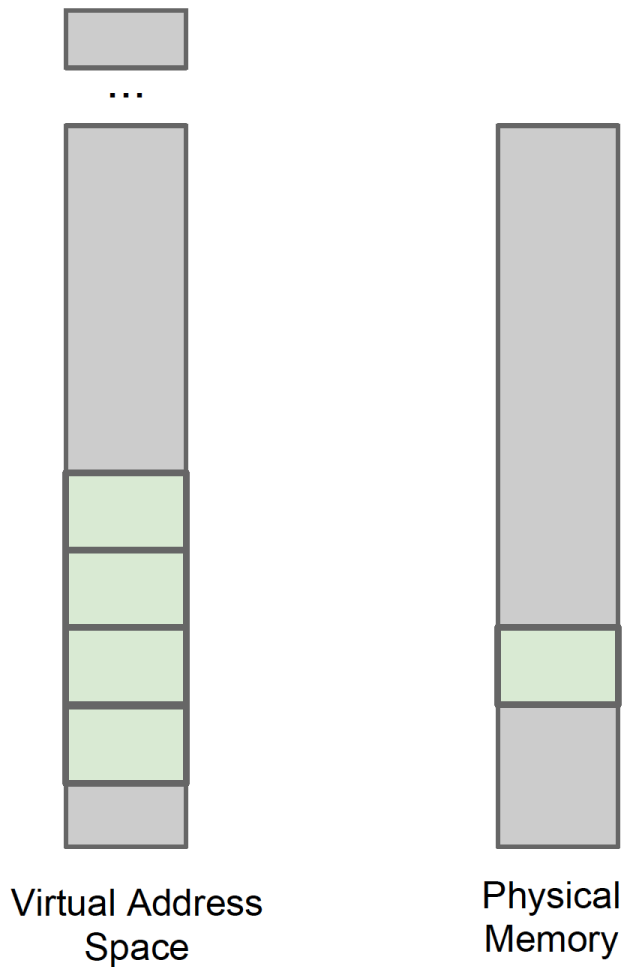
<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>



What happens when we map a file with read-write permissions? Indirection via page tables.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

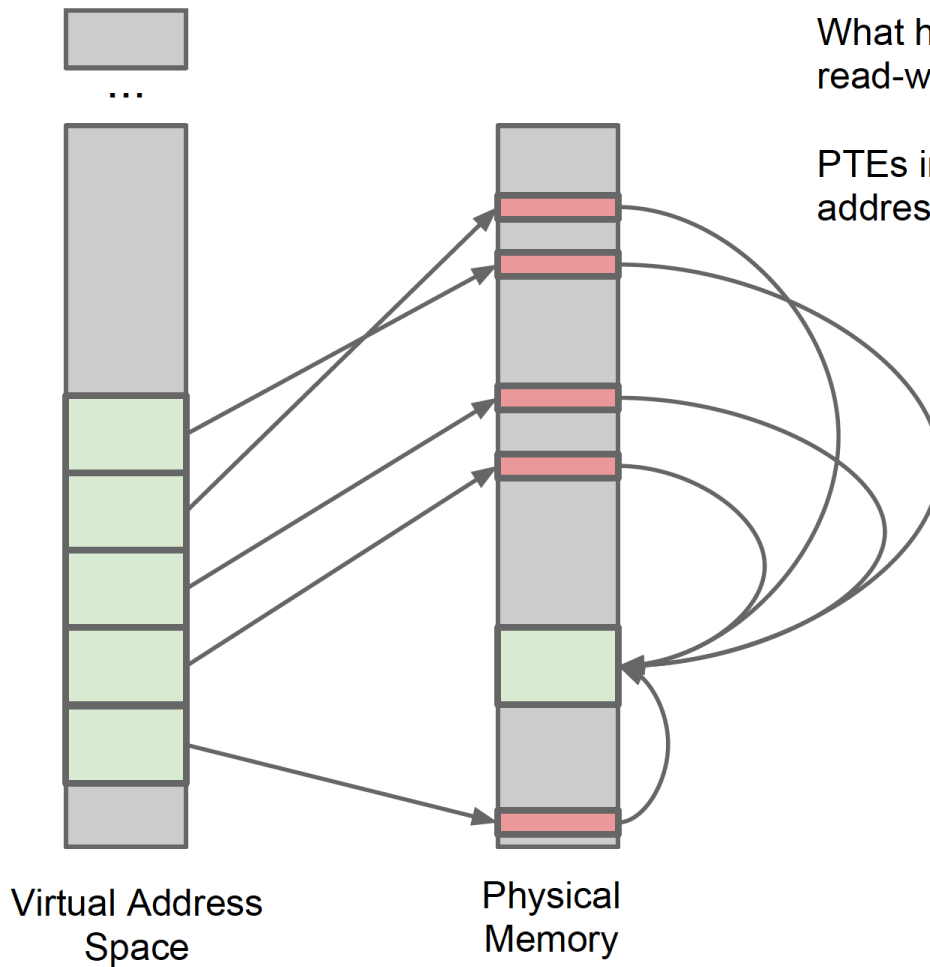
<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>



What happens when we repeatedly map a file with read-write permissions?

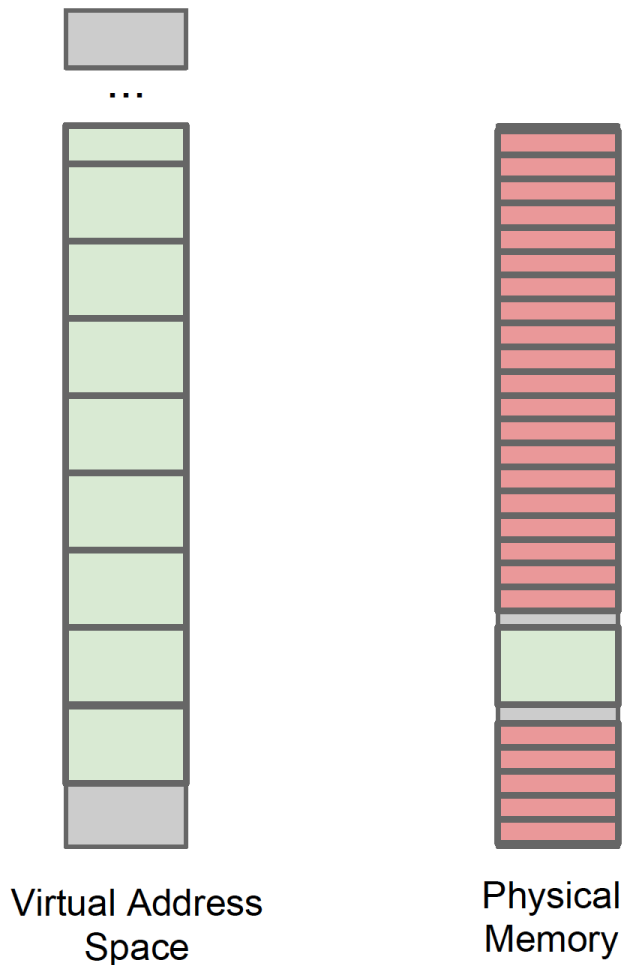
This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>



What happens when we repeatedly map a file with read-write permissions?

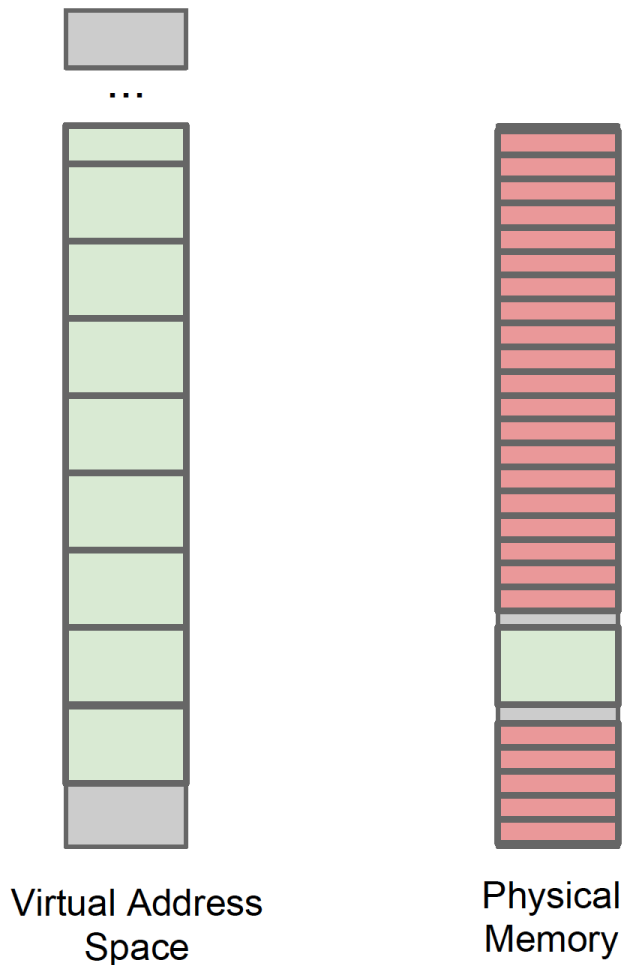
PTEs in physical memory help resolve virtual addresses to physical pages.



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

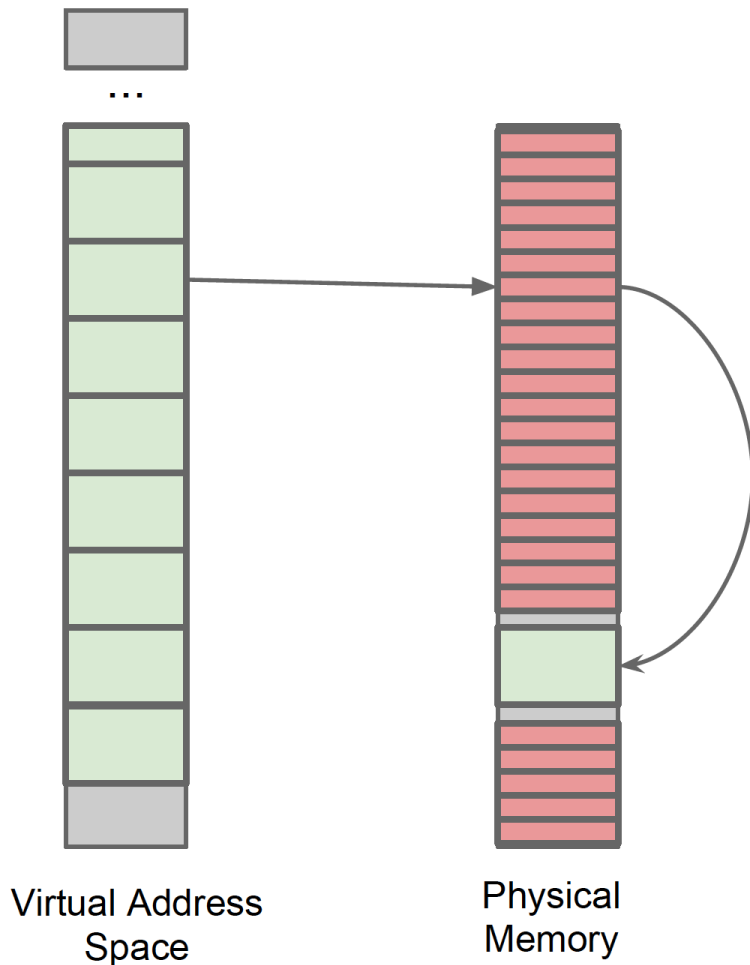


What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.



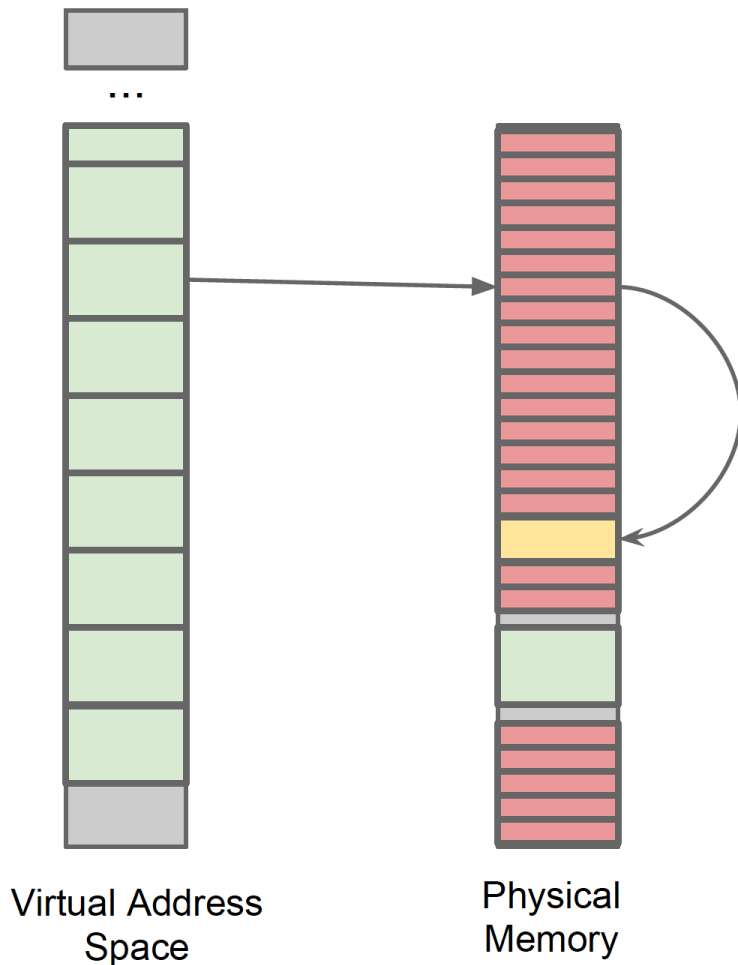
What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...



What happens when we repeatedly map a file with read-write permissions?

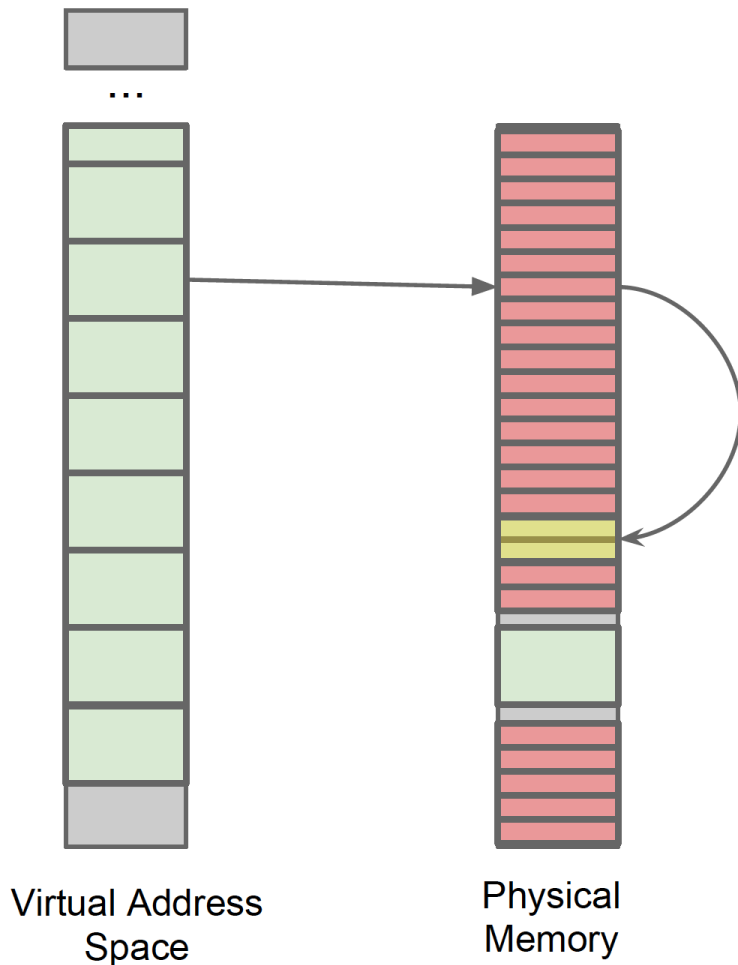
PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

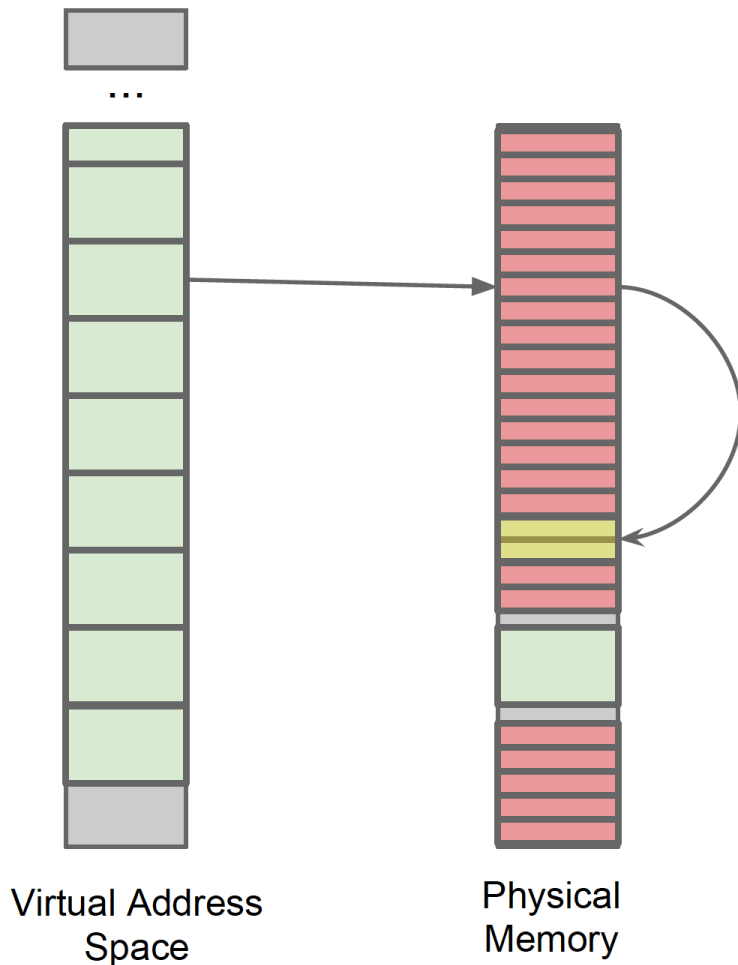
Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

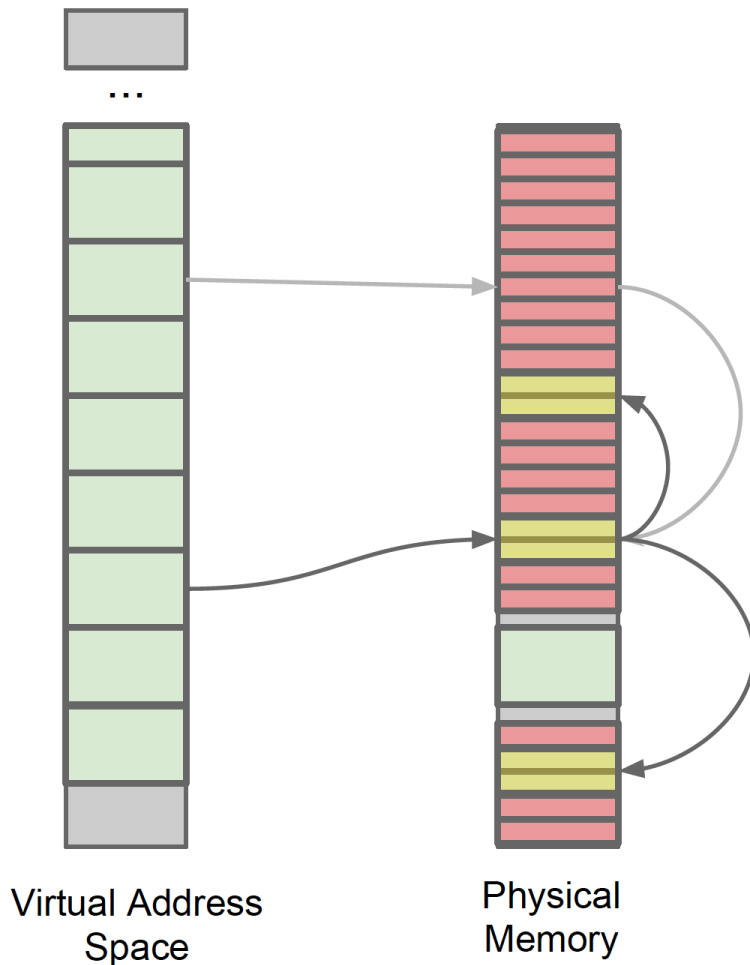
If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

An attacker that can read / write page tables ...

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

An attacker that can read / write page tables can use that to map **any** memory read-write.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

Exploit strategy

Privilege escalation in 7 easy steps ...

1. Allocate a large chunk of memory
2. Search for locations prone to flipping
3. Check if they fall into the “right spot” in a PTE for allowing the exploit
4. Return that particular area of memory to the operating system
5. Force OS to re-use the memory for PTEs by allocating massive quantities of address space
6. Cause the bitflip - shift PTE to point into page table
7. Abuse R/W access to all of physical memory

In practice, there are many complications.

This slide is from Mark Seaborn and Thomas Dullien's BlackHat 2015 talk

<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>

Some Issues in Virtual Memory

Three Major Issues in Virtual Memory

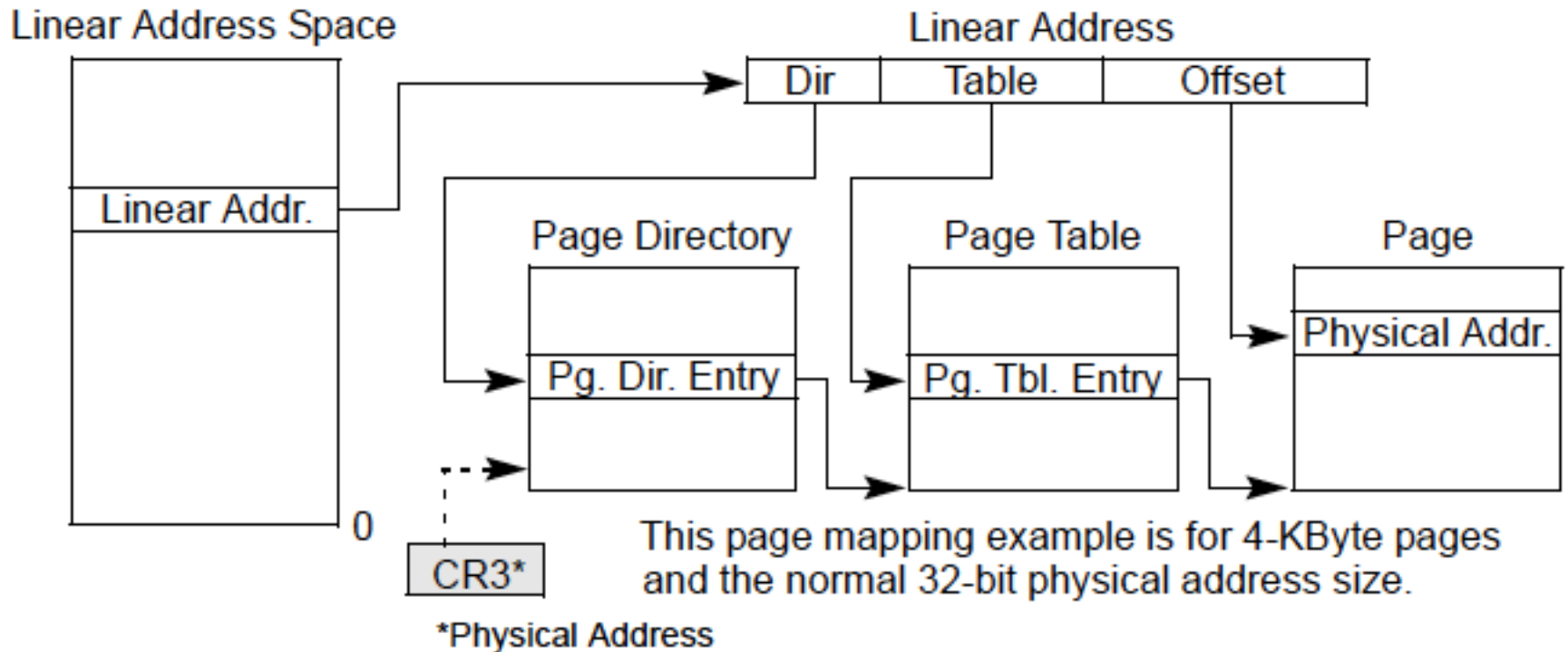
1. How large is the page table and how do we store and access it?
 2. How can we speed up translation & access control check?
 3. When do we do the translation in relation to cache access?
- There are many other issues we will not cover in detail
 - What happens on a context switch?
 - How can you handle multiple page sizes?
 - ...

Virtual Memory Issue I

- How large is the page table?
- Where do we store it?
 - ❑ In hardware?
 - ❑ In physical memory? (Where is the PTBR?)
 - ❑ In virtual memory? (Where is the PTBR?)
- How can we store it efficiently without requiring physical memory that can store all page tables?
 - ❑ **Idea: multi-level page tables**
 - ❑ Only the first-level page table has to be in physical memory
 - ❑ Remaining levels are in virtual memory (but get cached in physical memory when accessed)

Recall: Solution: Multi-Level Page Tables

Example from the x86 architecture



Page Table Access

- How do we access the Page Table?
- Page Table Base Register (CR3 in x86)
- Page Table Limit Register
- If VPN is out of the bounds (exceeds PTLR) then the process did not allocate the virtual page → access control exception
- Page Table Base Register is part of a process's context
 - Just like PC, status registers, general purpose registers
 - Needs to be loaded when the process is context-switched in

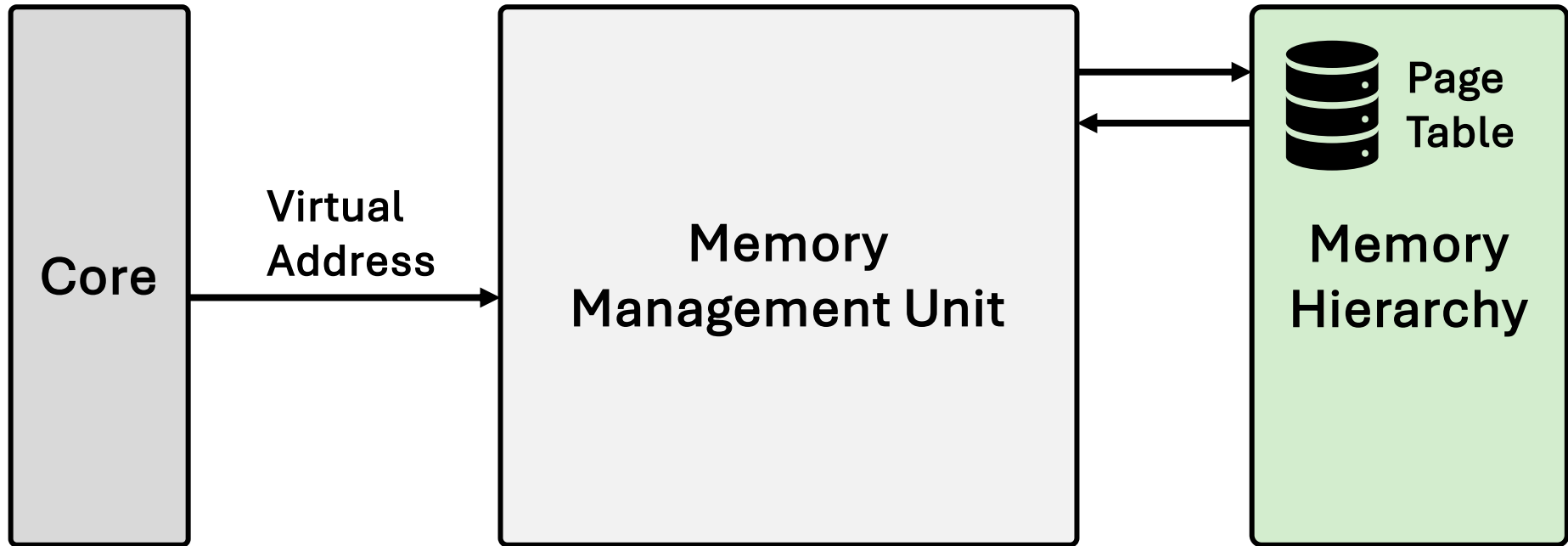
Three Major Issues in Virtual Memory

1. How large is the page table and how do we store and access it?
 2. How can we speed up translation & access control check?
 3. When do we do the translation in relation to cache access?
- There are many other issues we will not cover in detail
 - What happens on a context switch?
 - How can you handle multiple page sizes?
 - ...

Virtual Memory Issue II

- How fast is the address translation?
 - How can we make it fast?
- Idea: Use a hardware structure that caches PTEs and reduces the number of memory accesses to lookup the page table

Address Translation

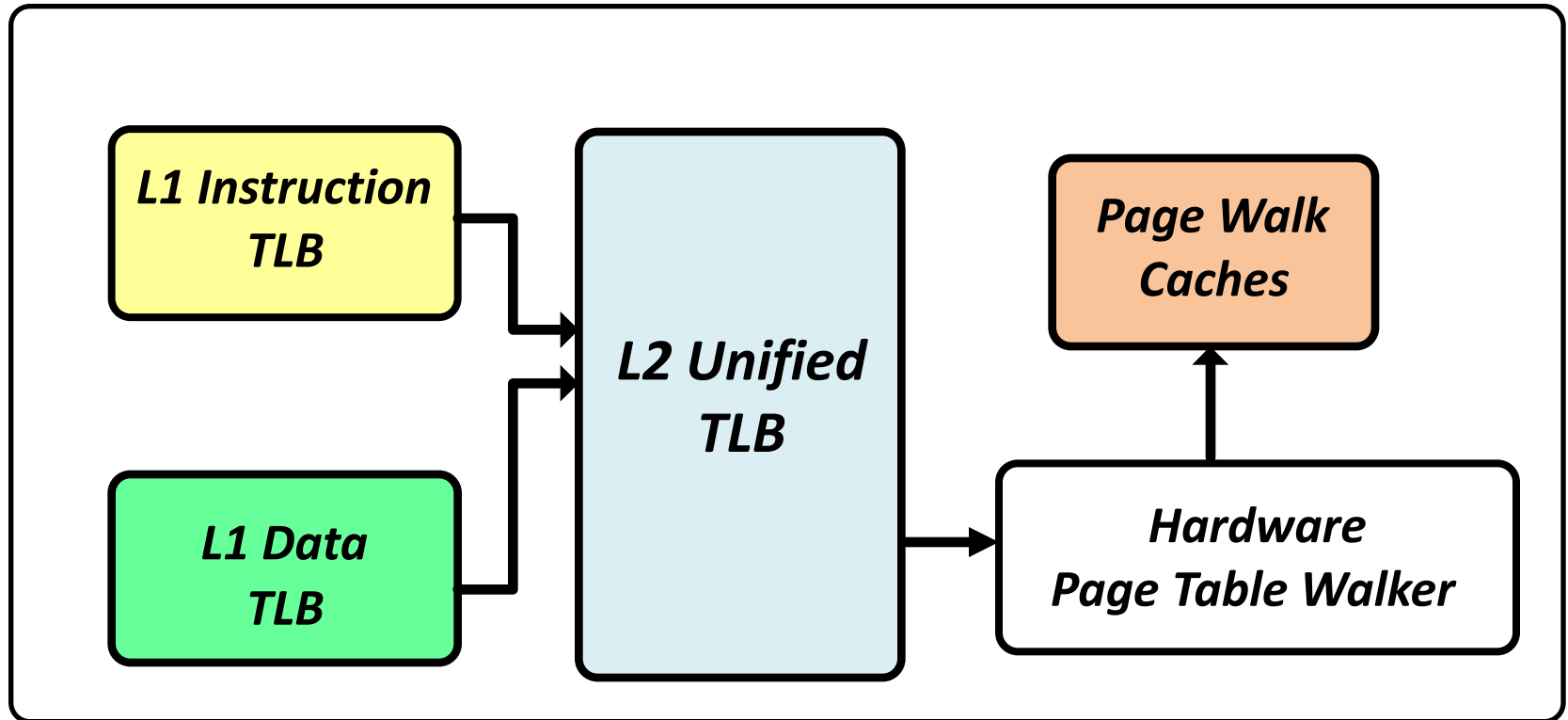


Hardware unit responsible
for address translation

Memory Management Unit

- The **Memory Management Unit (MMU)** is responsible for resolving address translation requests
 - One MMU per core (usually)
- MMU typically has three key components:
 - **Translation Lookaside Buffers** that cache recently-used virtual-to-physical translations (PTEs)
 - **Page Table Walk Caches** that offer fast access to the intermediate levels of a multi-level page table
 - **Hardware Page Table Walker** that sequentially accesses the different levels of the Page Table to fetch the required PTE

Intel Skylake: MMU



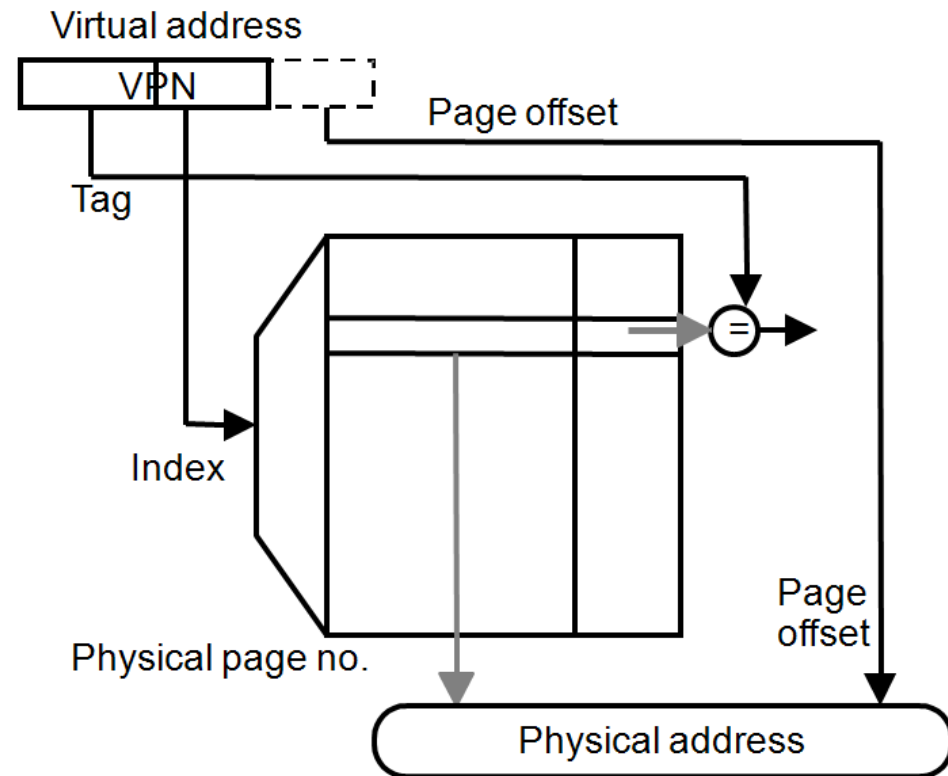
Translation Lookaside Buffer (TLB)

- Idea: Cache the Page Table Entries (PTEs) in a hardware structure in the processor to speed up address translation
- Translation lookaside buffer (TLB)
 - Small cache of most recently used Page Table Entries, i.e., recently used Virtual-to-Physical translations
 - Reduces the number of memory accesses required for *most* instruction fetches and loads/stores to only one TLB access

Speeding up Translation with a TLB

- A cache of address translations
 - Avoids accessing the page table on every memory access
- **Index** = lower bits of VPN (virtual page #)
- **Tag** = unused bits of VPN + process ID
- **Data** = a page-table entry
- **Status** = valid, dirty

The usual cache design choices (placement, replacement policy, multi-level, etc.) apply here too.



Handling TLB Misses

- The TLB is small; it cannot hold **all** PTEs
 - Some translation requests will inevitably miss in the TLB
 - Must access memory to find the required PTE
 - Called **walking the page table**
 - Large performance penalty
- Better TLB management & prefetching can reduce TLB misses
- Who handles TLB misses?
 - Hardware or software?

Handling TLB Misses (II)

- Approach #1. **Hardware-Managed** (e.g., x86)
 - The hardware does the **page walk**
 - The hardware fetches the PTE and inserts it into the TLB
 - If the TLB is full, the entry **replaces** another entry
 - Done transparently to system software
 - Can employ specialized structures and caches
 - E.g., page walkers and page walk caches

- Approach #2. **Software-Managed** (e.g., MIPS)
 - The hardware raises an exception
 - The operating system does the **page walk**
 - The operating system fetches the PTE
 - The operating system inserts/evicts entries in the TLB

Handling TLB Misses (III)

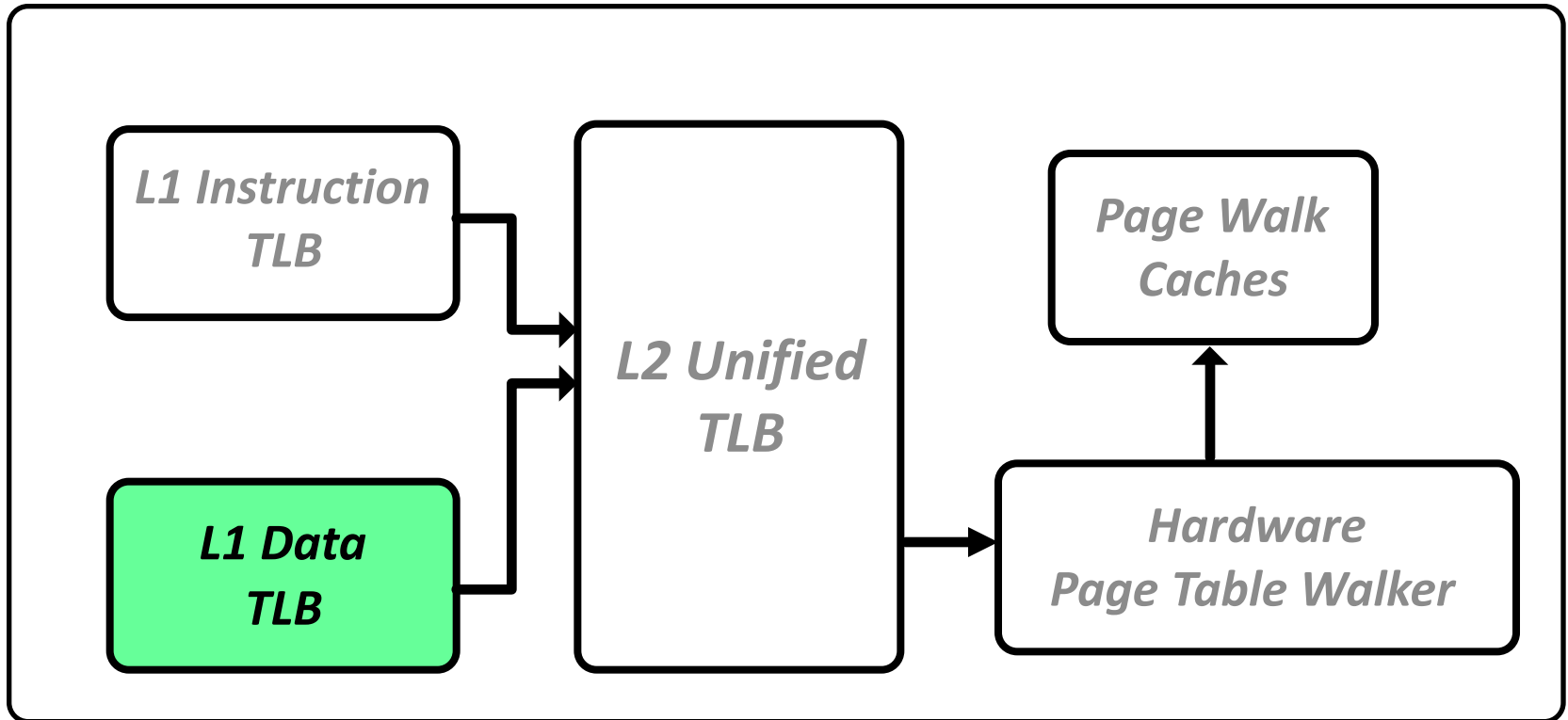
■ Hardware-Managed TLB

- + No exception on TLB miss. Instruction just stalls
- + Independent instructions may execute and help tolerate latency
- + No extra instructions/data brought into caches
- Page directory/table organization is fetched into the system:
OS has little flexibility in deciding these

■ Software-Managed TLB

- + The OS can define the page table organization
- + More sophisticated TLB replacement policies are possible
- Need to generate an exception → performance overhead due to pipeline flush, exception handler execution, extra instructions brought to caches

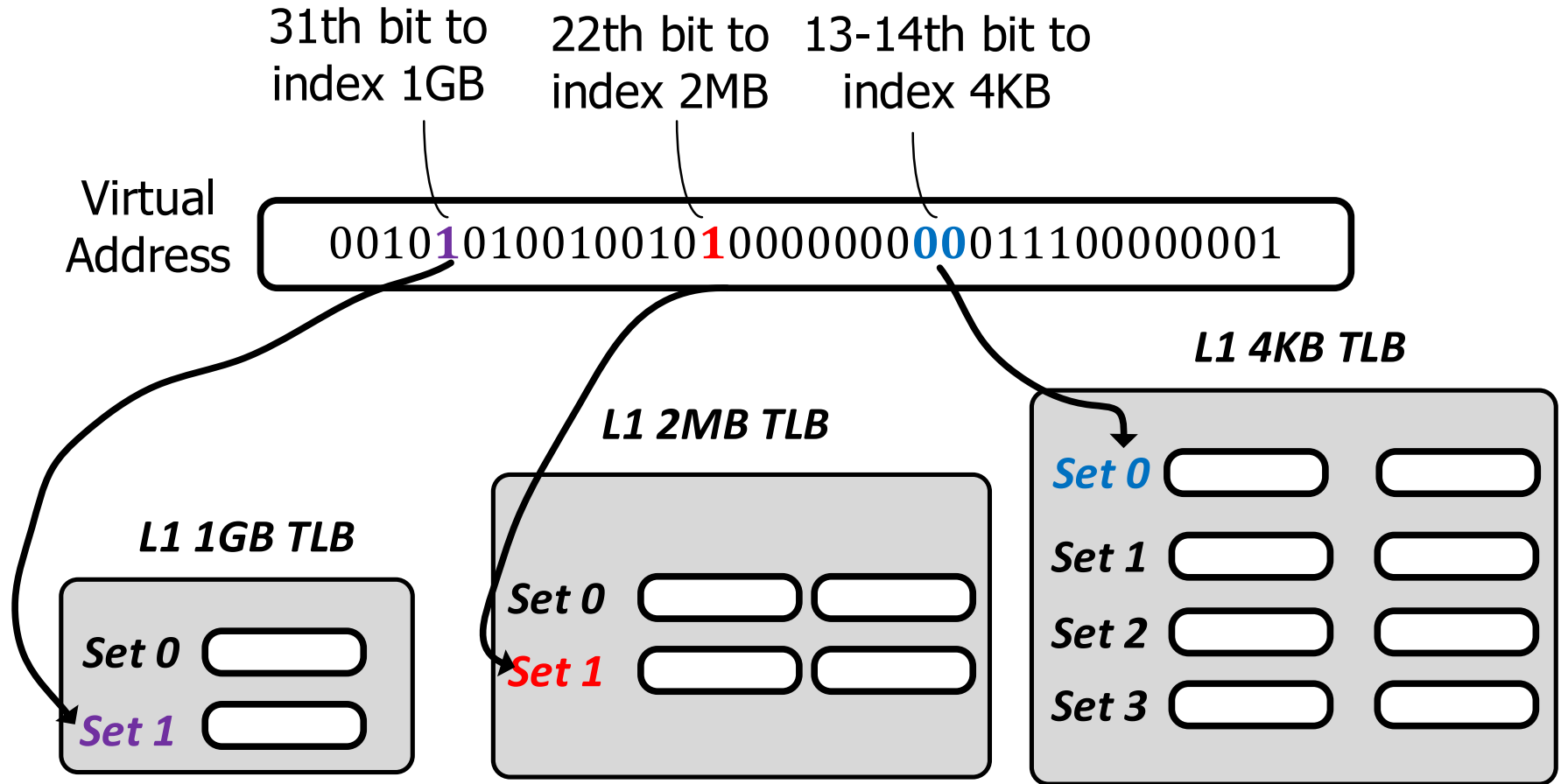
Intel Skylake: L1 Data TLB



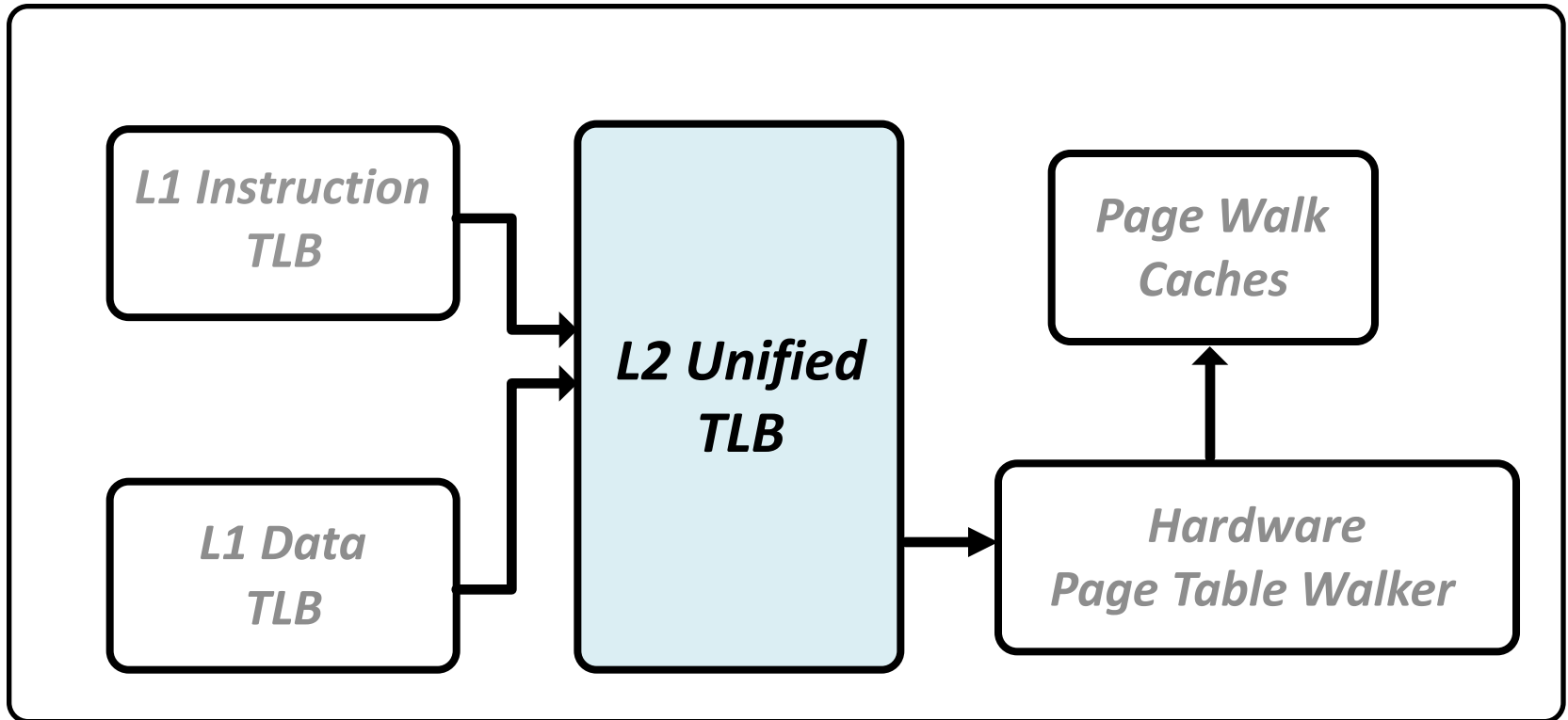
Intel Skylake: L1 Data TLB

- Separate L1 Data TLB structures for **4KB**, **2MB**, and **1GB** pages
- L1 DTLB
 - **4KB**: 64-entry, 4-way, 1 cycle access, 9 cycle miss
 - **2MB**: 32-entry, 4-way, 1 cycle access, 9 cycle miss
 - **1GB**: 4 entry, fully-associative
- Virtual-to-physical mappings are inserted in the corresponding TLB after a TLB miss
- During a translation request, all three L1 TLBs are looked up in parallel

L1 Data TLB: Parallel Lookup Example



Intel Skylake: L2 Unified I/D TLB



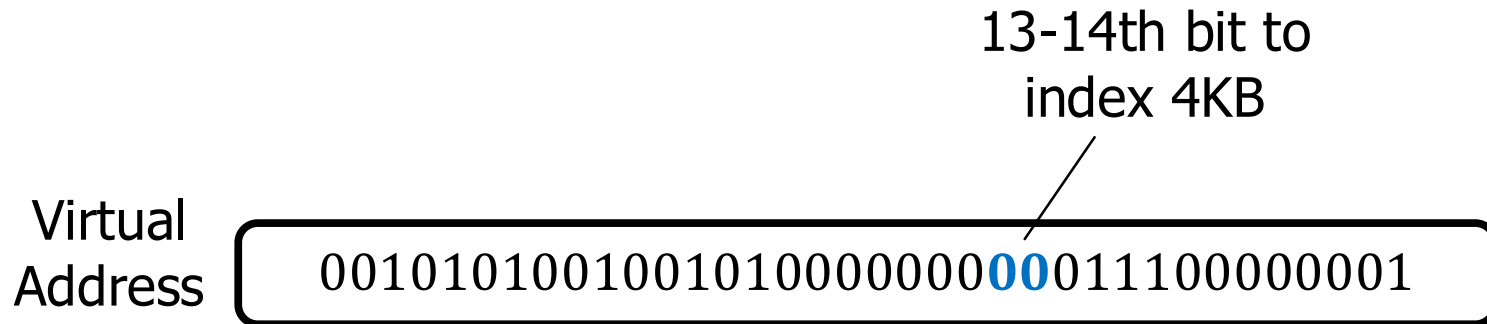
Intel Skylake: L2 Unified TLB

- L2 Unified TLB caches translations for both instr. and data
 - private per individual core
- 2 separate L2 TLB structures for 4KB/2MB and 1GB pages
- L2 TLB
 - **4KB/2MB**: 1536-entry, 12-way, 14 cycle access, 9 cycle miss
 - **1GB**: 16-entry, 4-way, 1 cycle access, 9 cycle miss penalty
- Challenge: How can the L2 TLB support both 4KB and 2MB pages using a single structure?
(Not enough publicly available information for Intel Skylake)

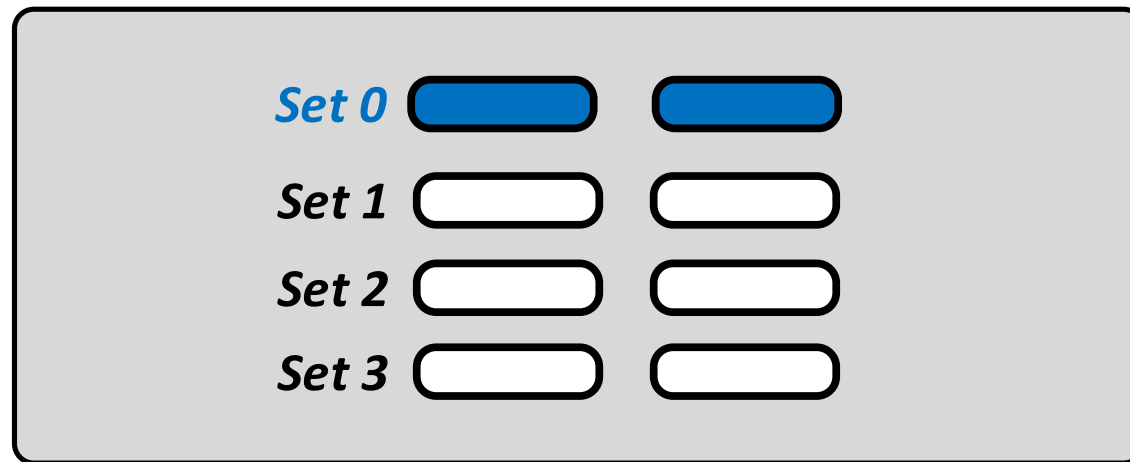
L2 Unified TLB: Accessing the TLB

- The 4KB/2MB structure of the L2 TLB is probed in 2 steps
- **Step 1:** Assume the page size is 4KB, calculate the index bits and access the L2 TLB
 - If the tag matches, it is a hit. If the tag does not match, go to Step 2.
- **Step 2:** Assume the page size is 2MB, **re-calculate** the index and access the L2 TLB.
 - If the tag matches, it is a hit. If the tag does not match, it is an L2 TLB miss.
- **General algorithm:**
 - Re-calculate index and probe TLB for all remaining page sizes

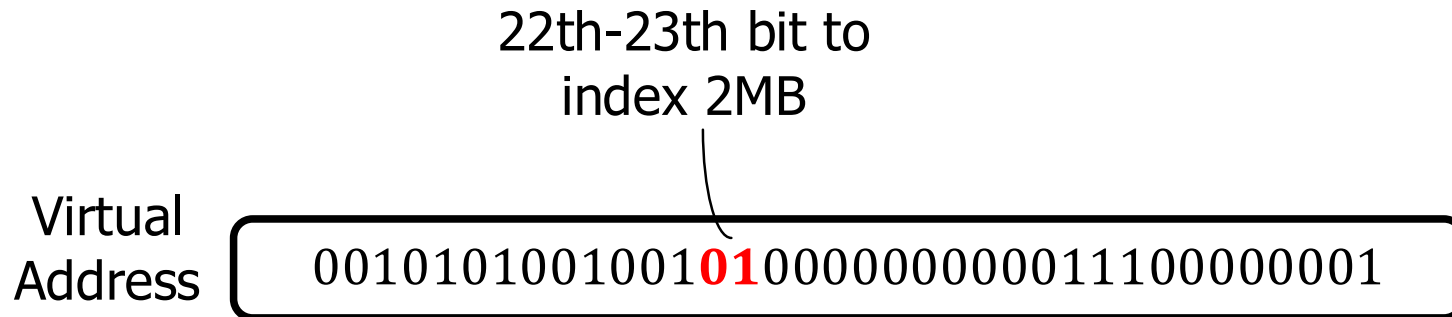
Step 1: Calculate Index for 4KB



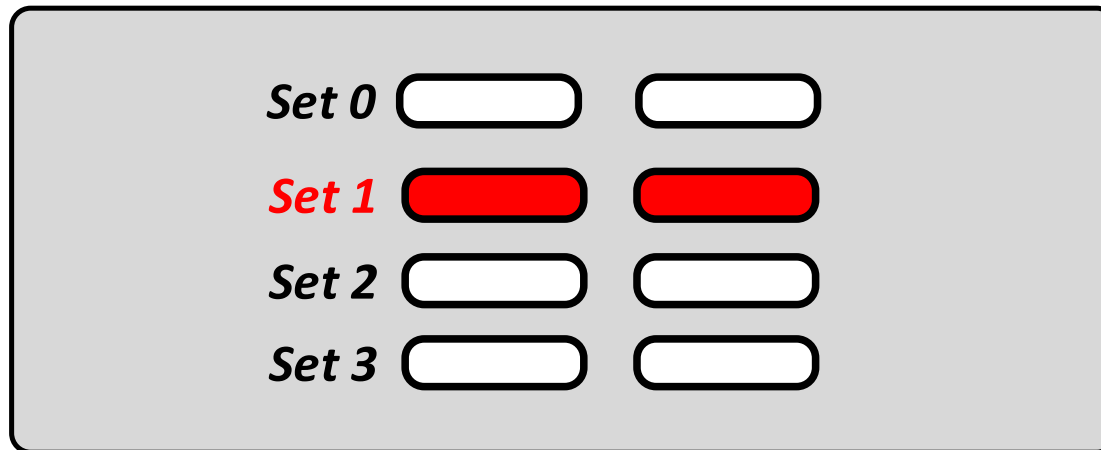
L2 TLB



Step 2: Re-calculate Index for 2MB



L2 TLB

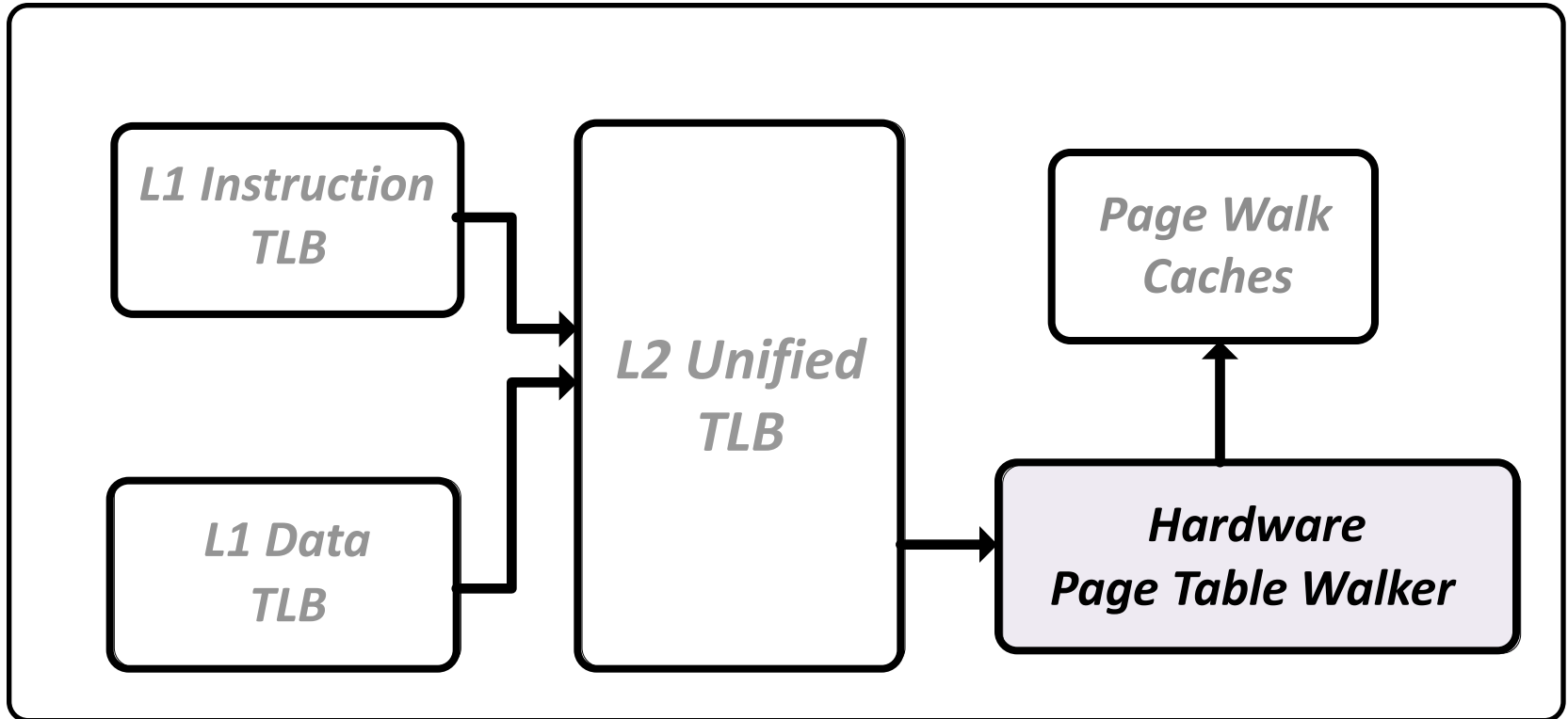


L2 TLB: N-Step Index Re-Calculation

- Pros:
 - + Simple and practical implementation
- Cons:
 - Varying L2 TLB hit latency (faster for 4KB, slower for 2MB)
 - Slower identification of L2 TLB Miss as all page sizes need to be tested
- Potential Optimizations:
 1. **Parallel Lookup:** Look up for 4KB and 2MB pages in parallel
 2. **Page Size Prediction:** Predict the probing order

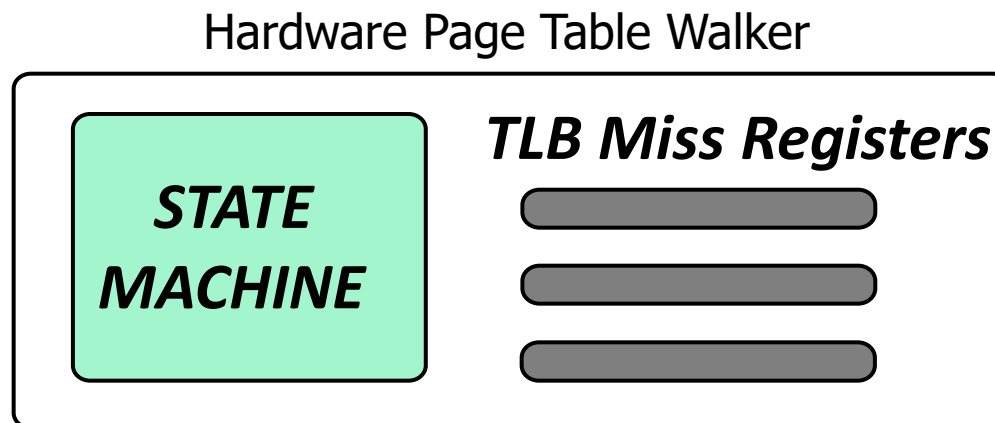
Tradeoffs are similar to “associativity in time” (also called pseudo-associativity)

Hardware Page Table Walker



Hardware Page Table Walker (I)

- A per-core hardware component that walks the multi-level page table to avoid expensive context switches & SW handling
- HW PTW consists of 2 components:
 - ❑ A state machine that is designed to be aware of the architecture's page table structure
 - ❑ Registers that keep track of outstanding TLB misses



Hardware Page Table Walker (II)

- Pros:

- + Avoids the need for context switch on TLB miss
- + Overlaps TLB misses with useful computation
- + Supports concurrent TLB misses

- Cons:

- Hardware area and power overheads
- Limited flexibility compared to software page table walk

Hardware Page Table Walker (III)

- PTW accesses the CR3 register that maintains information about the physical address of the root of the page table (PML4)
- PTW concatenates the content of CR3 with the first 9 bits of the virtual address

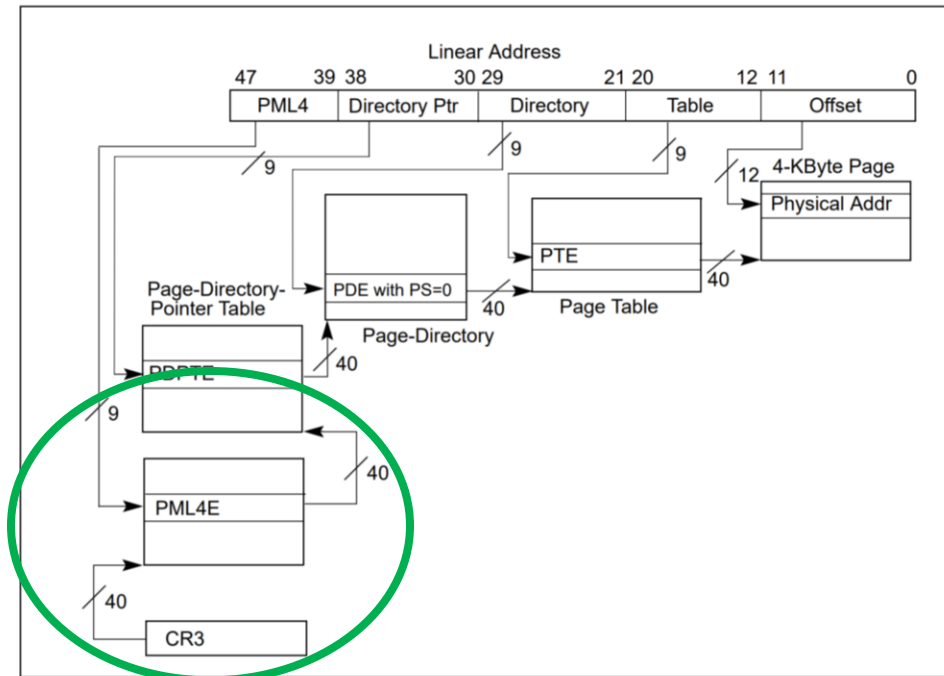


Figure 4-8. Linear-Address Translation to a 4-KByte Page using 4-Level Paging

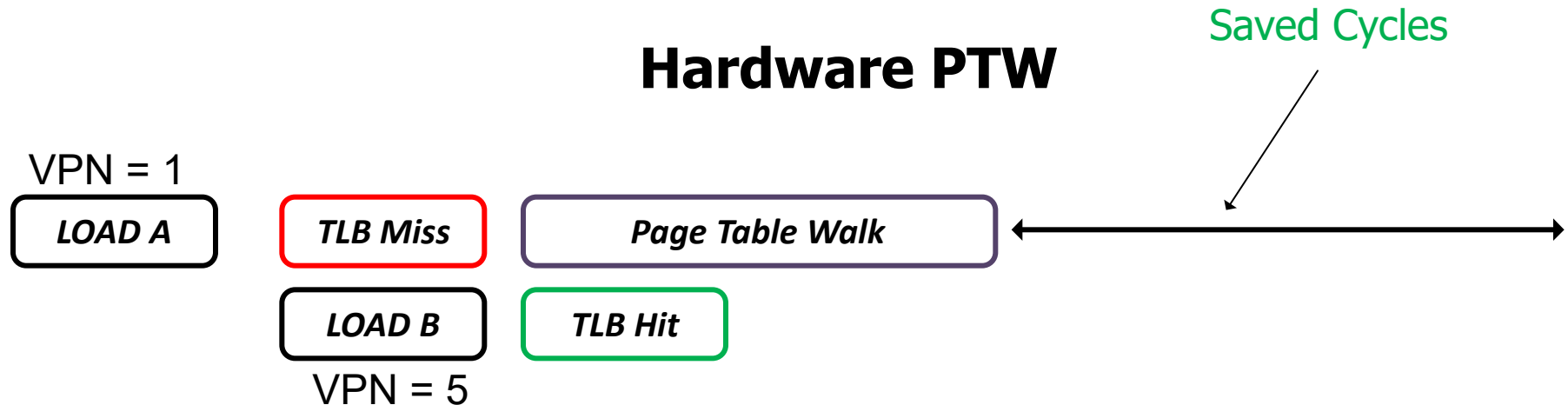
Hardware Page Table Walker (IV)

- Hardware PTWs allow overlapping TLB misses with useful computation

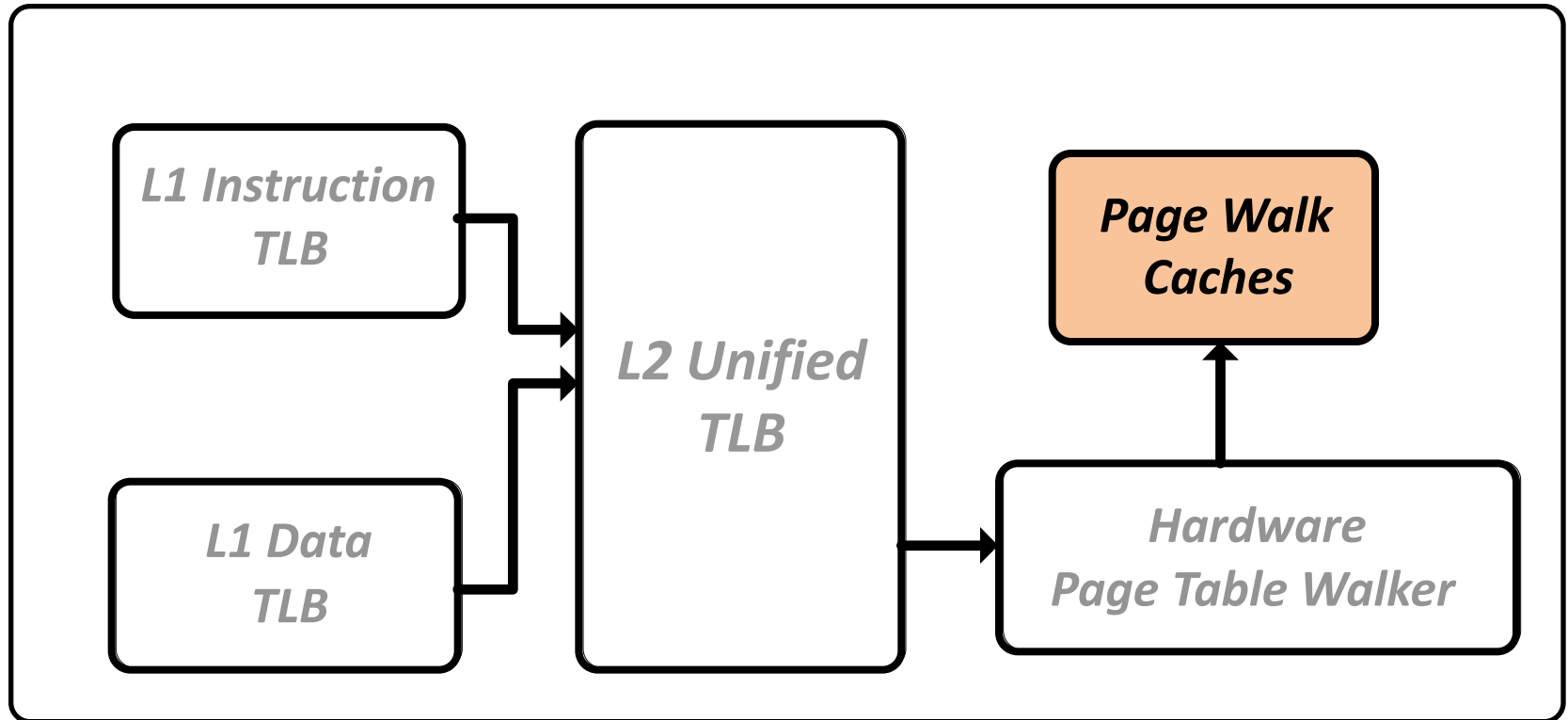
Software PTW



Hardware PTW



Page Walk Caches



Page Walk Caches

- Page Walk Caches cache translations from non-leaf levels of a multi-level page table to accelerate page table walks
- Page Walk Caches are low-latency caches that provide faster access to the page table levels
 - compared to accessing the regular cache/memory hierarchy for every page table walk

Three Major Issues in Virtual Memory

1. How large is the page table and how do we store and access it?
 2. How can we speed up translation & access control check?
 3. When do we do the translation in relation to cache access?
- There are many other issues we will not cover in detail
 - What happens on a context switch?
 - How can you handle multiple page sizes?
 - ...

Teaser: Virtual Memory Issue III

- When do we do the address translation?
 - Before or after accessing the L1 cache?

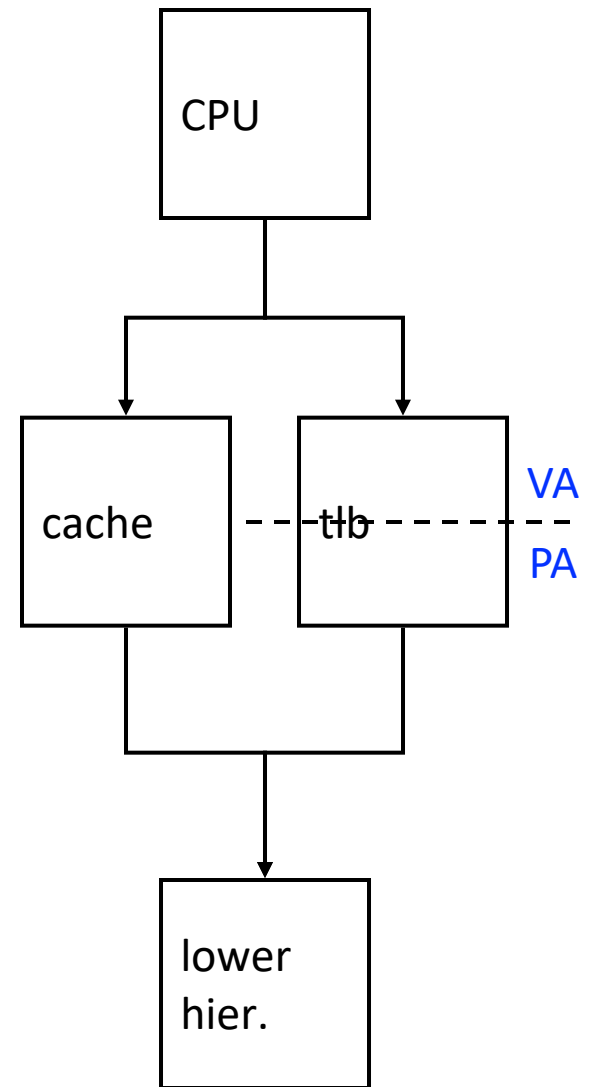
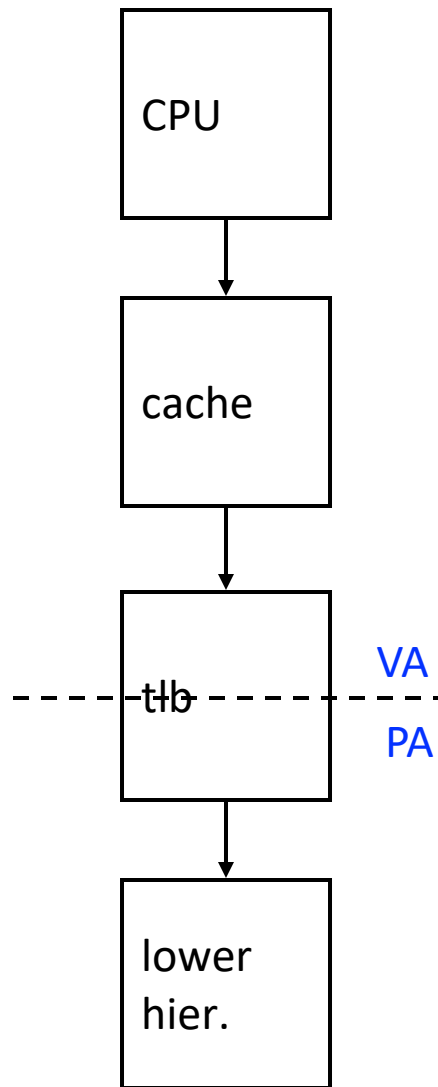
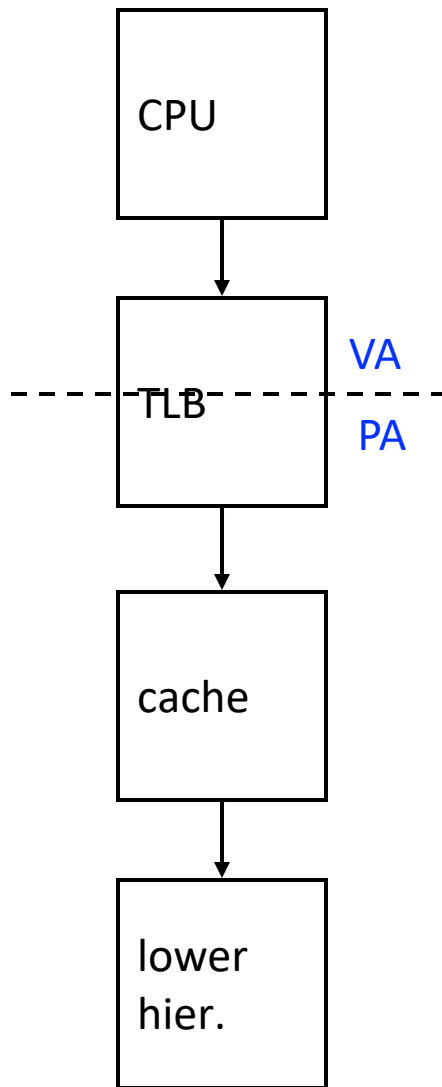
Address Translation and Caching

- When do we do the address translation?
 - Before or after accessing the L1 cache?
- In other words, is the cache virtually addressed or physically addressed?
 - Virtual versus physical cache
- What are the issues with a virtually addressed cache?
- **Synonym problem:**
 - Two different virtual addresses can map to the same physical address → same physical address can be present in multiple locations in the cache → can lead to inconsistency in data

Homonyms and Synonyms

- **Homonym: Same VA can map to two different PAs**
 - Why?
 - VA is in different processes
- **Synonym: Different VAs can map to the same PA**
 - Why?
 - Different pages can share the same physical frame within or across processes
 - Reasons: shared libraries, shared data, copy-on-write pages within the same process, ...
- Do homonyms and synonyms create problems when we have a cache?
 - Is the cache virtually or physically addressed?

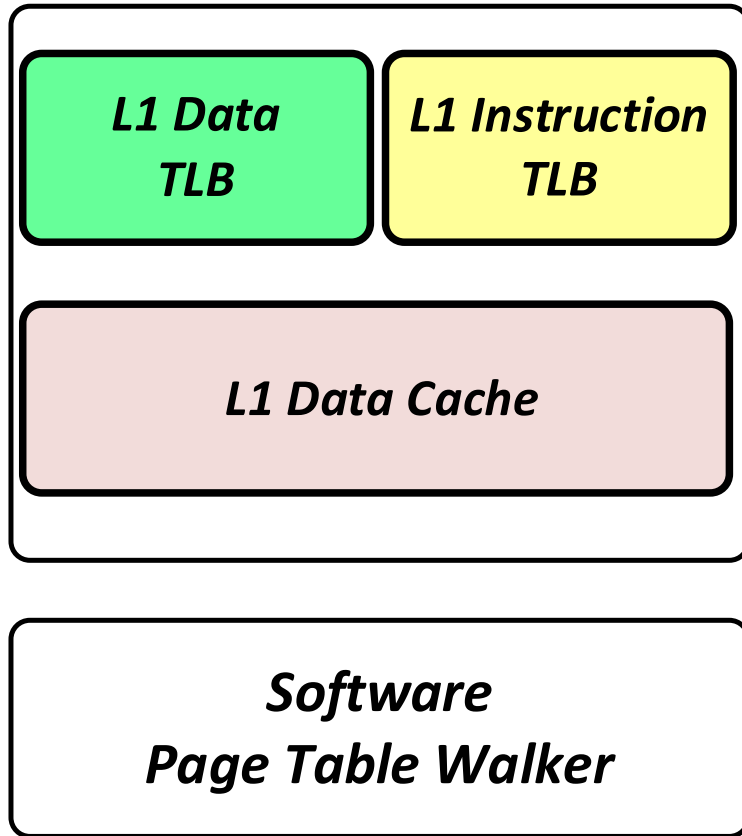
Cache-VM Interaction



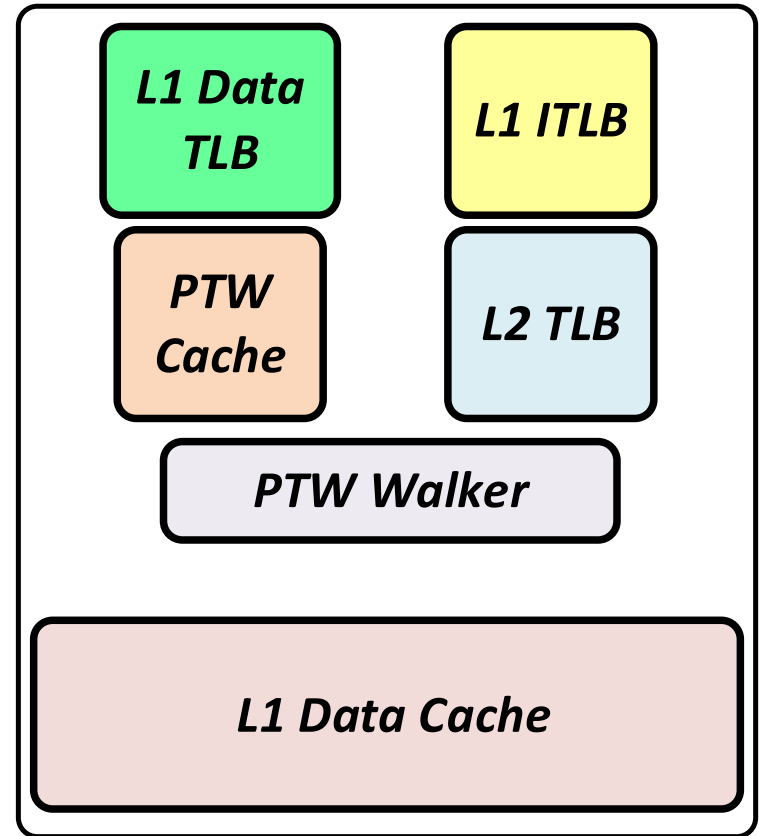
Evolution of the Virtual Memory System

Evolution of Address Translation

Simple Address Translation



Modern Address Translation



Modern Virtual Memory Designs

	A14 "Firestorm" (iPhone 12 Pro)	Intel/AMD/ARM
Decode width	8	4, 5 (Samsung M3), 5 (Cortex-X1)
ROB size	630	352 (Intel Willow Cove)
Load/store queue size	~148 outstanding loads ~106 outstanding stores	Intel Sunny Cove (128-LQ, 72-SQ) AMD Zen3 (64-LQ, 44-SQ)
L1-TLB	256 entries	64 entries
L2-TLB	3072 entries	1536 entries
Page size	16KB	4KB
L1-I cache	192KB	48KB (Intel Ice Lake)
L1-D cache	128KB, 3-cycles	32KB (Intel/AMD), 4-cycles
L2 cache	8MB shared across two big-cores, ~16-cycles	1MB (Intel Cascade Lake)
L3 cache	16MB shared across all CPU cores and integrated GPU	1.375 MB/core

Virtual Memory Overheads



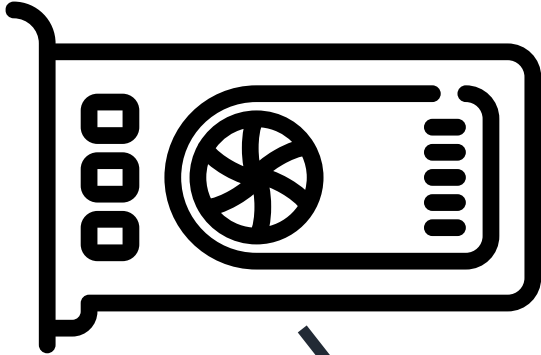
Enabling Fast and Accurate Virtual Memory Research with an Imitation-based Operating System Simulation Methodology

Presented at ASPLOS 2025

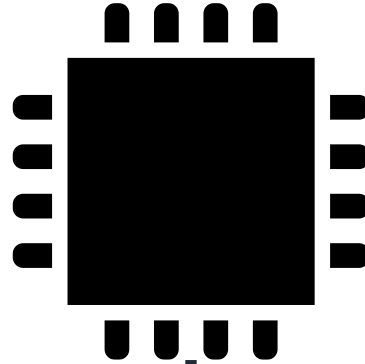
Konstantinos Kanellopoulos Konstantinos Sgouras F. Nisa Bostanci
Andreas Kosmas Kakolyris Berkin Kerim Konar Rahul Bera
Mohammad Sadrosadati Rakesh Kumar Nandita Vijaykumar Onur Mutlu

<https://github.com/CMU-SAFARI/Virtuoso>

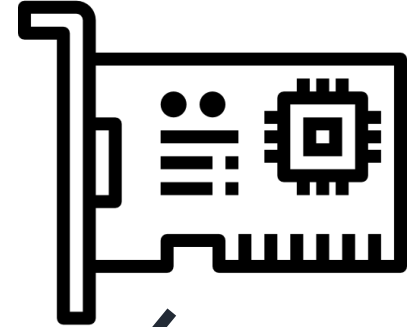
GPUs



CPUs



NICs



Virtual
Memory

Virtual Memory
is a cornerstone of
modern computing systems

**Does Virtual Memory
come for free?**

Virtual memory causes high **performance overheads**

**Memory
Allocation**

1

**Address
Translation**

2

Memory Allocation Overheads

1

Emerging Workloads



Function-as-a-Service

Gemini



Short-input Short-output
Large Language Model Inference

Short Running (<1s)

High spawning throughput

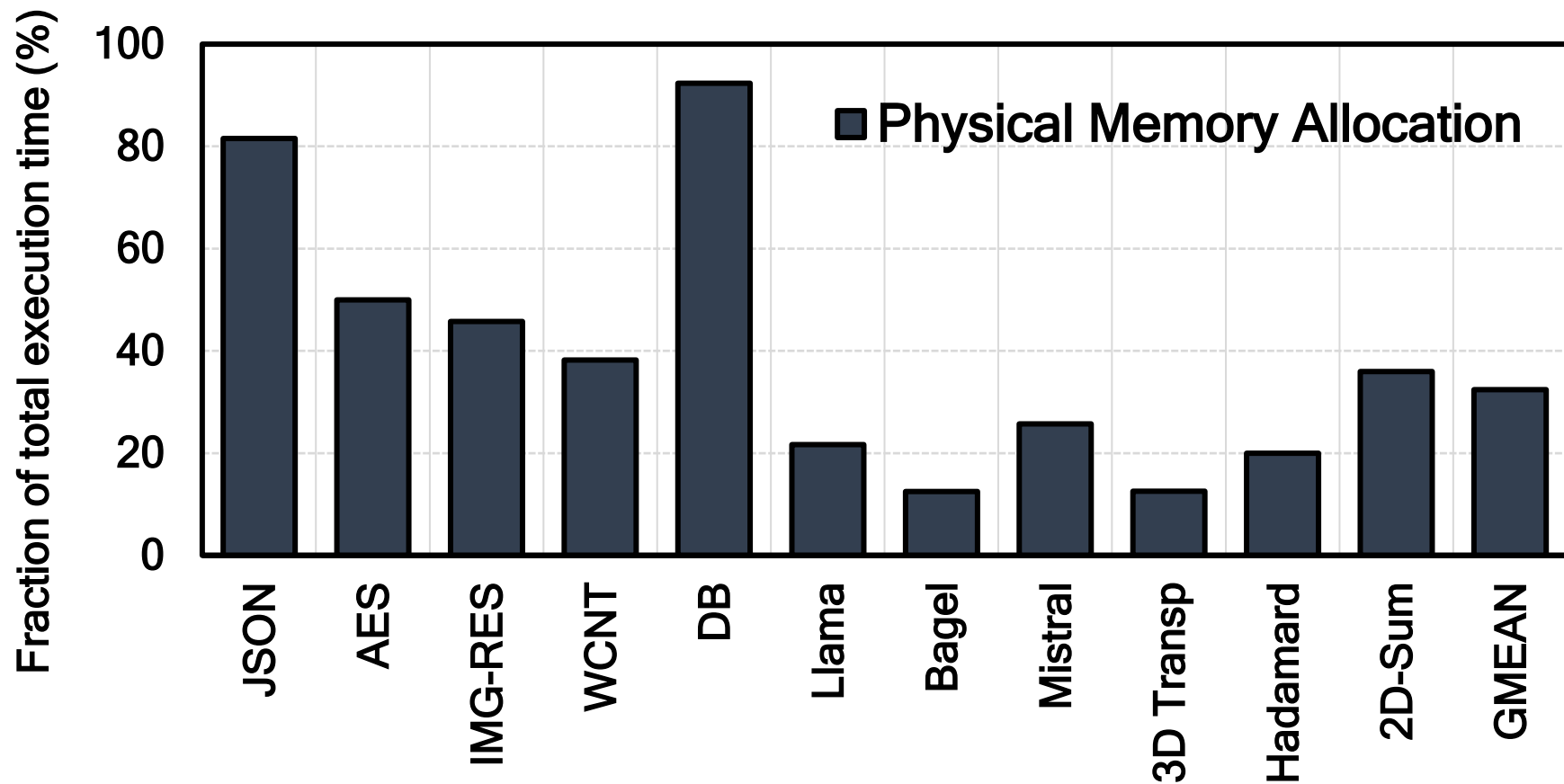
Time spent in OS cannot be amortized due to short execution

OS Memory
Allocation Routines

Execution

Time

Physical Memory Allocation Overheads



On average 32% of execution time (measured in a real system) spent on physical memory allocation

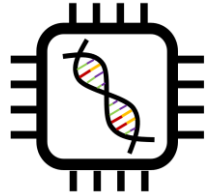
Address Translation Overheads

2

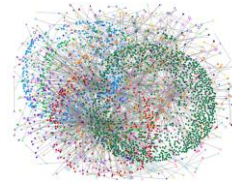
Emerging Workloads



Sparse Machine
Learning



Bioinformatics



Graph Analytics



Databases

Long Running (>100s) Irregular Memory Accesses

High Latency Address Translation

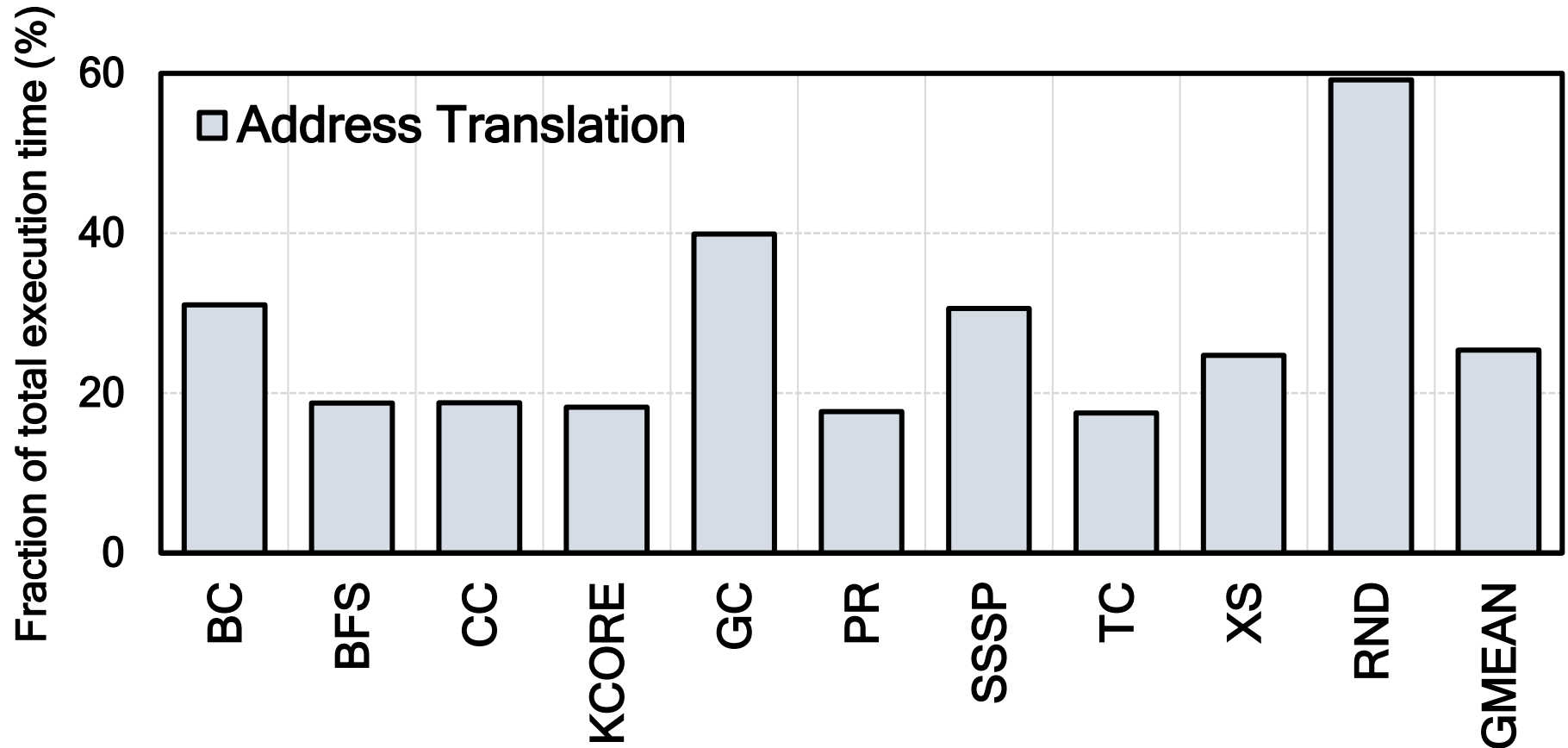
TLB Miss

Page Table Walk

Data Fetching

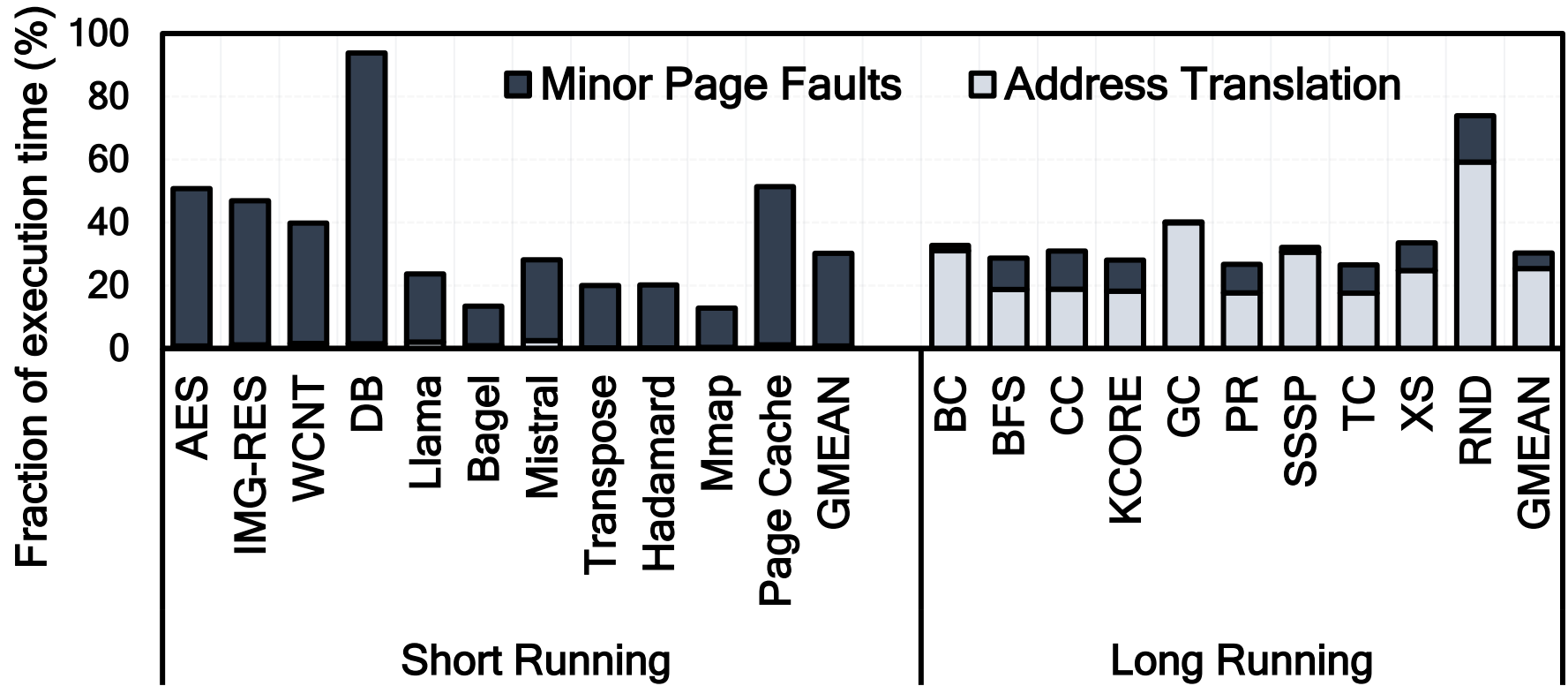
Time

Address Translation Overheads



On average 26% of execution time (measured in a real system) is spent on address translation

High VM Overheads

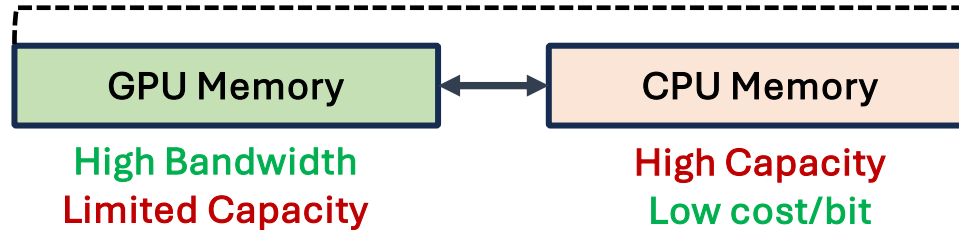


**VM causes high overheads
in diverse emerging workloads**

Virtual Memory overheads are **expected to increase** as we transition to larger physical address spaces

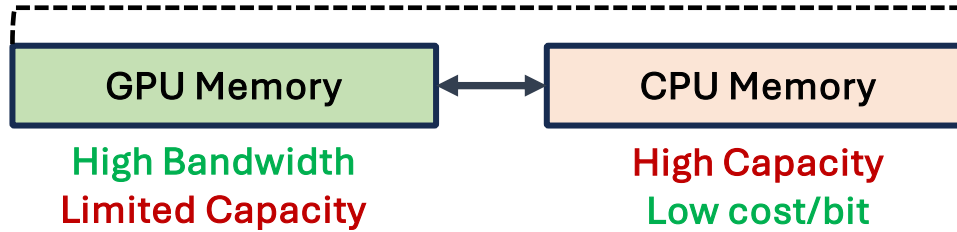
Going into the Future

1 Unified Virtual Memory

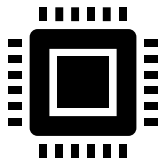


Going into the Future

1 Unified Virtual Memory



2 Direct Storage Access

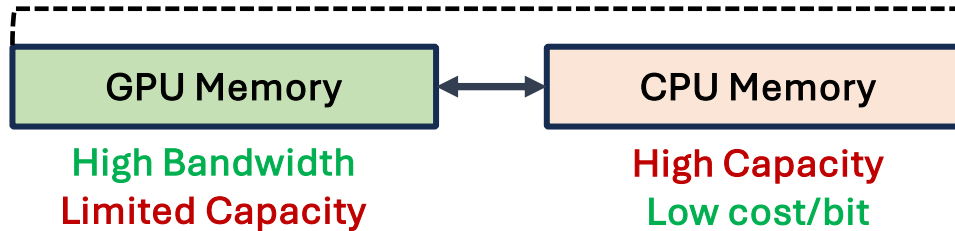


High-end SSD

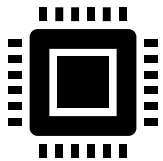
Byte-addressable
interface

Going into the Future

1 Unified Virtual Memory



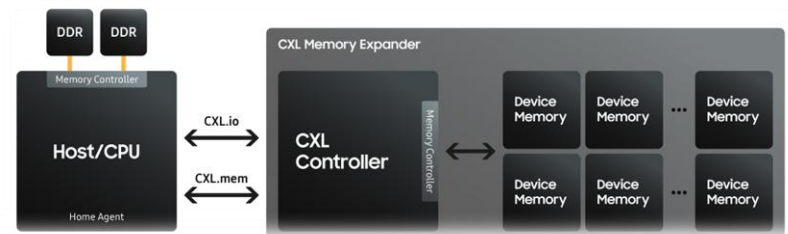
2 Direct Storage Access



High-end SSD

Byte-addressable
interface

Memory Disaggregation



Access to both local and remote
memory modules

Integration with 5 simulators

gem5-SE

Ramulator

Sniper

MQSim

ChampSim



Virtuoso

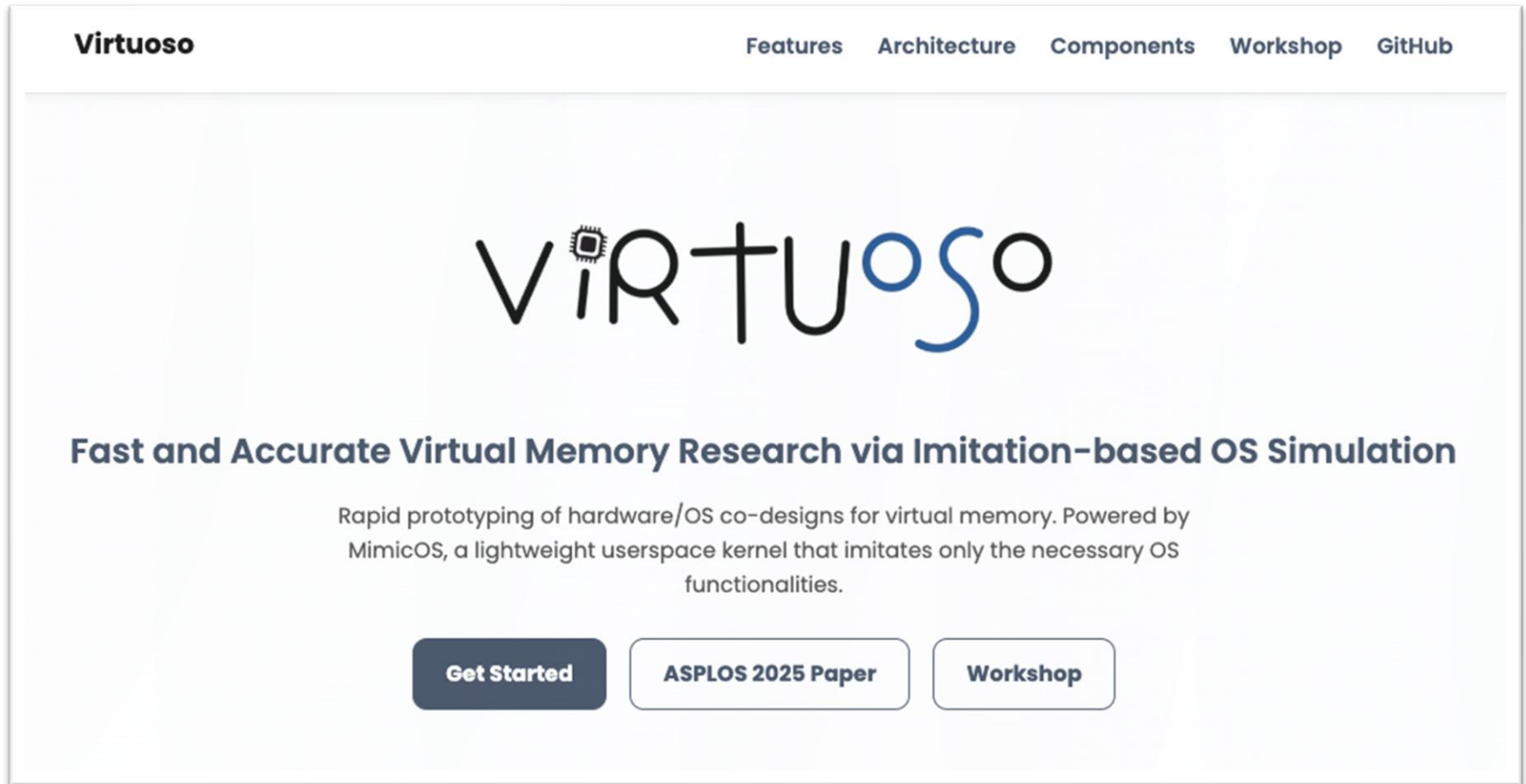
Toolset encompassing
multiple state-of-the-art
VM techniques

Validation against high-end server-grade CPU

Implemented 5 diverse use cases to
showcase Virtuoso's versatility

<https://github.com/CMU-SAFARI/Virtuoso>

Virtuoso Website – Getting Started



<https://github.com/CMU-SAFARI/Virtuoso>

Rethinking Virtual Memory (I)

Konstantinos Kanellopoulos, Hong Chul Nam, F. Nisa Bostanci, Rahul Bera, Mohammad Sadrosadati, Rakesh Kumar, Davide Basilio Bartolini, and Onur Mutlu,

"Victima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources"

Proceedings of the 56th International Symposium on Microarchitecture (MICRO), Toronto, ON, Canada, November 2023.

[[Slides \(pptx\)](#)] [[pdf](#)]

[[arXiv version](#)] [[Victima Source Code \(Officially Artifact Evaluated with All Badges\)](#)]

Officially artifact evaluated as available, functional, reusable and reproducible.

Distinguished artifact award at MICRO 2023

Victima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources

Konstantinos Kanellopoulos¹ Hong Chul Nam¹ F. Nisa Bostanci¹ Rahul Bera¹
Mohammad Sadrosadati¹ Rakesh Kumar² Davide Basilio Bartolini³ Onur Mutlu¹

¹ETH Zürich ²Norwegian University of Science and Technology ³Huawei Zurich Research Center

Rethinking Virtual Memory

Konstantinos Kanellopoulos, Rahul Bera, Kosta Stojiljkovic, Nisa Bostanci, Can Firtina, Rachata Ausavarungnirun, Rakesh Kumar, Nastaran Hajinazar, Mohammad Sadrosadati, Nandita Vijaykumar, and Onur Mutlu,

"Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings"

Proceedings of the 56th International Symposium on Microarchitecture (MICRO),
Toronto, ON, Canada, November 2023.

[[Slides \(pptx\)](#) ([pdf](#))]

[[arXiv version](#)]

[[Utopia Source Code](#)]

Utopia: Fast and Efficient Address Translation via Hybrid Restrictive & Flexible Virtual-to-Physical Address Mappings

Konstantinos Kanellopoulos¹ Rahul Bera¹ Kosta Stojiljkovic¹ Nisa Bostanci¹ Can Firtina¹
Rachata Ausavarungnirun² Rakesh Kumar³ Nastaran Hajinazar⁴ Mohammad Sadrosadati¹
Nandita Vijaykumar⁵ Onur Mutlu¹

¹ETH Zürich ²King Mongkut's University of Technology North Bangkok

³Norwegian University of Science and Technology ⁴Intel Labs ⁵University of Toronto

Rethinking Virtual Memory

Nastaran Hajinazar, Pratyush Patel, Minesh Patel, Konstantinos Kanellopoulos, Saugata Ghose, Rachata Ausavarungnirun, Geraldo Francisco de Oliveira Jr., Jonathan Appavoo, Vivek Seshadri, and Onur Mutlu, **"The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework"**

Proceedings of the 47th International Symposium on Computer Architecture (ISCA), Virtual, June 2020.

[[Slides \(pptx\)](#) ([pdf](#))]

[[Lightning Talk Slides \(pptx\)](#) ([pdf](#))]

[[ARM Research Summit Poster \(pptx\)](#) ([pdf](#))]

[[Talk Video](#) (26 minutes)]

[[Lightning Talk Video](#) (3 minutes)]

[[Lecture Video](#) (43 minutes)]

The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework

Nastaran Hajinazar^{*†} Pratyush Patel[⌘] Minesh Patel^{*} Konstantinos Kanellopoulos^{*} Saugata Ghose[‡]
Rachata Ausavarungnirun[⊙] Geraldo F. Oliveira^{*} Jonathan Appavoo[◇] Vivek Seshadri[▽] Onur Mutlu^{*‡}

^{*}ETH Zürich [†]Simon Fraser University [⌘]University of Washington [‡]Carnegie Mellon University

[⊙]King Mongkut's University of Technology North Bangkok [◇]Boston University [▽]Microsoft Research India

Virtual Memory Summary

Virtual Memory Summary

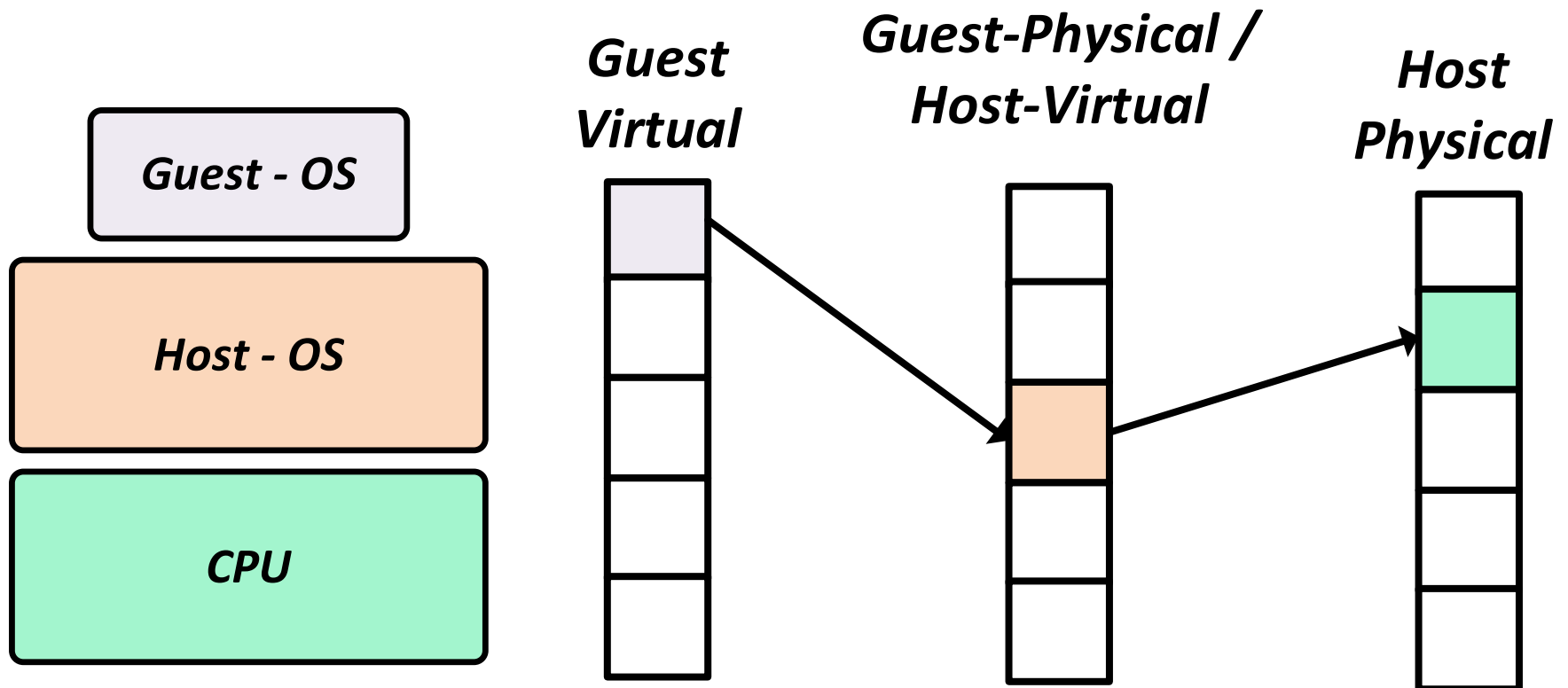
- Virtual memory gives the illusion of “infinite” capacity
- A subset of virtual pages are located in physical memory
- A page table maps virtual pages to physical pages – this is called address translation
- A TLB speeds up address translation
- Multi-level page tables keep the page table size in check
- Using different page tables for different programs provides memory protection

There is More... We Will Not Cover...

- How to handle virtualized systems?
 - Virtual machines running programs
 - Hypervisors
- ...

Virtual Memory in Virtualized Environments

- Virtualized environments (e.g., Virtual Machines) need to have an additional level of address translation



Virtual Memory: Parting Thoughts

- VM is one of the most successful examples of
 - ❑ architectural support for programmers
 - ❑ how to partition work between hardware and software
 - ❑ hardware/software cooperation
 - ❑ programmer/architect tradeoff
- Going forward: How does virtual memory scale into the future? Four key trends:
 - ❑ Increasing, huge physical memory sizes (local & remote)
 - ❑ Hybrid physical memory systems (DRAM + NVM + SSD)
 - ❑ Many accelerators in the system addressing physical memory
 - ❑ Virtualized systems (hypervisors, software virtualization, local and remote memories)

Lectures on Virtual Memory

Challenges

- **Three examples** of the **challenges** in adapting conventional virtual memory frameworks for increasingly-diverse systems:
 - Requiring a **rigid page table structure**
 - High address **translation overhead** in virtual machines
 - **Inefficient** heterogeneous memory **management**



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 12c: The Virtual Block Interface (ETH Zürich, Fall 2020)

726 views • Oct 31, 2020

16 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Virtual Memory

Address Translation Overhead

*High latency
PTWs*

+

*Frequent
PTWs*



*High performance
overheads*

*High interference in
memory hierarchy*



1:47:09 / 2:47:07

SAFARI



Onur Mutlu Lectures

41,8 χιλ. εγγεγραμμένοι



Εγγραφήκατε



23



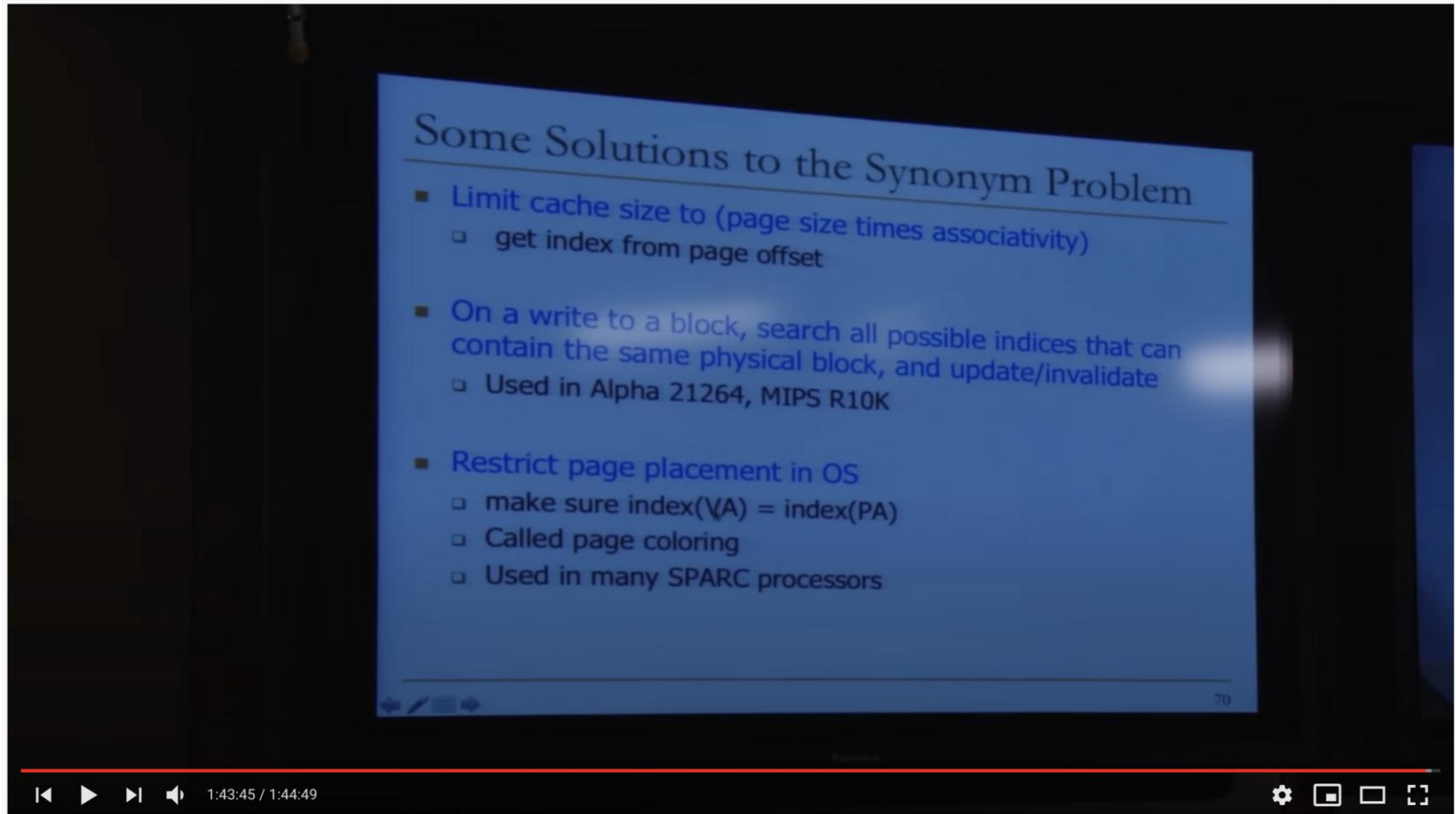
Κοινοποίηση



Λήψη



Lectures on Virtual Memory



Some Solutions to the Synonym Problem

- Limit cache size to (page size times associativity)
 - get index from page offset
- On a write to a block, search all possible indices that can contain the same physical block, and update/invalidate
 - Used in Alpha 21264, MIPS R10K
- Restrict page placement in OS
 - make sure $\text{index}(\text{VA}) = \text{index}(\text{PA})$
 - Called page coloring
 - Used in many SPARC processors

Lecture 20. Virtual Memory - Carnegie Mellon - Comp. Arch. 2015 - Onur Mutlu

22,313 views • Mar 7, 2015

139 5 SHARE SAVE ...



Carnegie Mellon Computer Architecture
23.3K subscribers

SUBSCRIBED



Lectures on Virtual Memory

■ Computer Architecture, Spring 2015, Lecture 20

- Virtual Memory (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=2RhGMpY18zw&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=22>

■ Computer Architecture, Fall 2020, Lecture 12c

- The Virtual Block Interface (ETH, Fall 2020)
- <https://www.youtube.com/watch?v=PPR7YrBi7IQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=24>

■ Computer Architecture, Fall 2023, Lecture 23

- Utopia, Victima, Virtuoso (ETH, Fall 2023)
- https://www.youtube.com/watch?v=teb8jc3U1JY&list=PL5Q2soXY2Zi9UFWwfRtSjpKX_IRF0IPSe&index=23

Digital Design & Computer Arch.

Lecture 26: Virtual Memory

Konstantinos Kanellopoulos

Prof. Onur Mutlu

ETH Zürich

Spring 2026

29 May 2026